CHAPTER-1

INTRODUCTION TO ORGANIZATION

SOLITAIRE INFOSYS is a leading Software and Web Application Development Company, based in Mohali (Chandigarh) that provides high quality comprehensive services to enterprises across a wide range of platforms and technologies. Our major areas of expertise are in providing quality, cost effective software or web development.

Figure1.1 Head office

1.1 Focus of the Company

The Company focuses on understanding the diverse and mission-critical needs of each of our clients. To understand is to be able to deliver. The competence and experience of our company gives us a competitive edge by making sure we provide the best services and products to our clients. Our high quality standards enable us to deliver reliable and error-free software applications, despite their complexity.

The Company provides Web design/Web development, B2B & B2C Ecommerce solutions, SEO & Web Promotions strategies implementation consulting services to both domestic as well as international clients at the most affordable rates.

Operating Excellence

- A safe, fun and professional work environment
- Good relationships with industry and other partners
- Respect for the rights and ambitions of our employees
- Honorable, value-driven business relations

1.2 History Of The Company

Solitaire Infosys Inc. was a dream came into existence over three years ago with a strong aspiration of becoming a best IT service provider around the globe. Presently, Solitaire Infosys is already leading the race with its competitors. Being nurtured by a team of experienced and sensitive people. We try to bond emotionally with our clients and love to go an extra mile to satisfy their needs, which is the reason that we hold the edge in the league.

At Solitaire Infosys Inc. we provide the experience, expertise and capabilities that enable organizations to accelerate their service processes, deliver more service value and realize breakthrough results in the markets they serve. Now our clients not only appreciate our comprehensive range of services, our innovative, streamlined and cost effective solution, but more importantly, they appreciate our high level of excellent customer service which is unparalleled in the industry. Solitaire Infosys Pvt. Ltd. envisions becoming a reliable partner to all its clients and focusing on doing everything ethically and rightly. We are always open to accept our mistakes and have the nerve to do the necessary changes. They have the experience, expertise, and capabilities to enable organizations to accelerate their service processes in every possible way.

CHAPTER 2

INTRODUCTION TO PROJECT

Tomato is a user-friendly food app designed to make ordering your favorite mealsa breeze. Built using the MERN stack (MongoDB, Express.js, React, and Node.js), Tomato offers a smooth and fast experience for users. With a simple interface, you can easily browse menus, place orders, and track deliveries right from your phone. Whether you're craving pizza, sushi, or a healthy salad, Tomatoconnects you with local restaurants and delivers delicious food straight to your door. Enjoy the convenience of great meals with just a few taps!

2.1 Why the MERN Stack?

The choice of the MERN stack for Tomato's development plays a crucial role in its performance and functionality.

React, the front-end library, allows for the creation of dynamic and responsive user interfaces. It enables real-time updates, so users can see their order status without refreshing the page. This enhances user engagement and satisfaction, as customers can track their deliveries in real-time.

Finally, Node.js provides a powerful runtime environment for building the server-side logic of the app. Its event-driven architecture ensures that Tomato can efficiently manage numerous connections, making it ideal for a busy fooddelivery service.

2.2 User Experience

Tomato is designed with the user in mind. The app features a clean and simple interface, making navigation effortless. Users can easily search for their favoritedishes, filter options based on dietary preferences.

2.3 The Need for a Food App Like Tomato

In our busy lives today, many people are looking for convenient ways to order food, and that's where apps like Tomato come in. Tomato is designed to make iteasy for anyone to enjoy their favourite meals without the hassle of cooking or going out. With just a few taps on a smartphone, users can browse menus from various local restaurants, customize their orders, and pay securely—all in one place.

One of the biggest reasons people love food apps is the convenience they offer. Whether you're at home, at work, or on the go, you can quickly find a meal that suits your cravings. Tomato allows you to explore a wide range of cuisines, fromcomfort food to healthy options, so everyone can find something they like. This variety is especially important for families and busy individuals who may not have time to cook or decide where to eat.

Another important factor is time-saving. In a world where everyone seems to be in a hurry, having a food app can really help. Instead of waiting in long lines at arestaurant or cooking at home, you can place your order in advance and have it delivered right to your door. This is especially useful during lunch breaks or aftera long day when the last thing you want to do is spend time preparing a meal.

Tomato also offers real-time order tracking, which enhances the user experience.Thanks to the technology behind the app, users can see exactly where their orderis—from when it's being prepared to when it's out for delivery. This not only keeps customers informed but also builds trust, making them more likely to use the app again in the future.

The technology behind Tomato, known as the MERN stack, plays a key role in making all of this possible. It helps manage user data, restaurant menus, and orders efficiently. This means that users can easily access their order history andget personalized recommendations based on their preferences. The app is designed to handle many users at once, ensuring that everyone

has a smooth experience, even during busy times.

Moreover, food apps like Tomato benefit not just users but also local restaurants.By featuring smaller eateries on the platform, Tomato helps them reach a wider audience and increase their sales. This connection fosters a sense of community,encouraging users to explore new dining options and support local businesses.

2.4 Features of Tomato: A Food App Using MERN

Tomato is designed to make ordering food easy and enjoyable. Here are some of its key features that make it stand out:

User-Friendly Interface: Tomato has a simple and clean design, making it easy for anyone to navigate. You can quickly find what you're looking for without any confusion.

Menu Browsing: Users can easily browse menus from various local restaurants. You can see all the available dishes, along with pictures and prices, helping you make the best choice.

Order Customization: You can customize your orders based on your preferences. Whether you want extra toppings or specific dietary options, Tomato allows you to tailor your meal to your liking.

Secure Payment Options: Tomato offers multiple secure payment methods, including credit/debit cards and digital wallets. This ensures that your transactions are safe and convenient.

Real-Time Order Tracking: After placing an order, you can track its status in real-time. You'll receive updates on when your food is being prepared and when it's on its way, so you're never left in the dark.

Order History: Tomato keeps a record of your past orders, making it easyto reorder your favourites with just a few taps. This feature is great for those busy days when you know what you want.

User Reviews and Ratings: You can read reviews and ratings from othercustomers before trying a new restaurant or dish. This helps you make informed decisions and discover great new places to eat.

Special Offers and Promotions: Tomato regularly features special deals and discounts from local restaurants. This gives you the chance to enjoy your favourite meals at a lower price.

Personalized Recommendations: Based on your previous orders and preferences, Tomato can suggest new dishes and restaurants you might like. This makes exploring new food options easy and fun.

Support for Local Restaurants: Tomato promotes local eateries, giving them a platform to reach more customers. This helps support the community and encourages users to try local flavours.

2.5 Scope of the Project

The scope of an online food ordering project would involve designing and developing comprehensive and user-friendly web or mobile application that enables customers to order food from a wide range of restaurants while also providing restaurants with powerful tools for managing orders and deliveries. The Tomato food app project using the MERN stack will focus on buildinga platform for users to order food easily. Users will be able to browse a variety of restaurants and view detailed menus that include descriptions, prices, and images of each dish. The app will feature a simple search function, allowing users to quickly find specific items or restaurants. Creating a personal account will enhance the user experience, enabling features like saving favourite restaurants, managing payment methods, andstoring delivery addresses.

The project would need to include a front-end user interface that provides a seamless and intuitive experience for customers, enabling them to easily browse menus, select items and place orders. This would require designing a visually appealing and user-friendly interface using web development technologies such as HTML, CSS and JavaScript as well as integrating with restaurant APIs to retrieve and display menu data.

On the back-end, the application would need to efficiently process and manage orders, handles payments and coordinate with delivery partners. The ordering process will be straightforward, with users adding items to acart and customizing their orders. Secure payment options will be integrated to facilitate easy transactions. Once an order is placed, users willreceive real-time updates on its status, including notifications for order confirmation, preparation, and delivery tracking.

Users will also have the opportunity to leave reviews and ratings after receiving their orders, helping others make informed decisions. This feedback system will allow users to share their experiences and view the opinions of others. React will be used to build an interactive and dynamicuser interface. The app will be designed with a responsive layout, ensuringit works well on both mobile devices and desktops. User interface and experience will be prioritized, with thoughtful design and accessibility features included.

- To provide the security Low cost Free delivery your order.
- Available for 24*7 hours.

Processing Environment

Admin can see all the users who registered on his/her website. He can also see all the orders placed by the users. The users have to register on the website to place any order.

- How Registration Works?
- First of all, users Registers themselves with the Site.

- Then after registration they can login on the website.
- They can see the products and there details and can place the order.

Acceptance Criteria:

Acceptance criteria are an important yet, in my experience, often overlooked or undervalued aspect of the iterative planning process. Acceptance criteria are super important because projects succeed or fail based on the ability of the team to meet their customers documented and perceived acceptance criteria. When we clearly define acceptance criteria up front, we avoid surprises at the end of a sprint, or release, and ensure a higher level of customer satisfaction. Acceptance criteria should be written from the perspective of the end-user or customer. That may seems obvious since user stories are written from the perspective of the end-user or customer. However, even for the best-written stories, acceptance criteria are where the development and testing perspective seems to creep in.

Acceptance Criteria may reference what is in the project's other User Stories or design documents to provide details, but should not be a re-hash of them. They should be relatively high-level while still providing enough detail to be useful.

CHAPTER 3

PROBLEM FORMATION AND OBJECTIVES

3.1 Existing system

In the context of a **Food App** using mern, an existing system would mean a fully or partially functional application that integrates MongoDB, Express.js, React, and Node.js to perform the core tasks of a food ordering and delivery platform.

This system might already include key features like:

- User sign-up/login.
- Restaurant browsing and menu display.
- Cart management and order checkout.
- Backend operations for processing orders and payments.
- Real-time order tracking.
- Admin tools for managing restaurants, orders, and analytics.

An existing system could either be a real-world application (like those deployed on platforms such as GitHub, or used by a specific company) or a sample project built as part of learning or prototyping in MERN development

3.2 Literature Review

Online food ordering systems have become increasingly popular in recent years. Here are some research papers that may be helpful for a literature survey on food app:

1. “A Study of Food App”: By N.Ashar, P.Dhoke, and N. Deo this paper examines different types of food app and analysis their features, benefits and drawbacks.

2. “A Food App for Restaurants”: By A.G. Abdelkhalek and M.S.Mohmoud. This paper proposes a food app that allows customers to place orders from different restaurant using a single platform.

3. “Development of Online Food System”: By R. Ahmed, M.U. Ahmed and M.S.Islam. This paper provides a detailed review of different online food systems, their features and their performance.

4. “A Comprehensive review of Food App”: By A.Jain and A.Agagrwal. This paper design and development of online food app that includes features such as menu management, ordering and payment process.

5. “User Acceptance of Online Food Ordering App: An empirical study” By A.Mishra and R.L. Roy. This paper examines the factors that affect user acceptance of online food ordering app including usefulness, ease of use and system security.

3.3 Objectives

The objectives of Tomato are to make ordering food easy, diverse, and enjoyable while supporting local businesses and creating a sense of community. By focusing on what users want, Tomato strives to create a great food experience foreveryone. Tomato is all about making it easy and fun for people to order food. One of its main goals is convenience, so users can quickly find and order their favourite meals whenever they want. The app features a variety of options from local restaurants, helping people explore different types of food and discover new favourites. Personalization is another focus for Tomato. The app suggests new dishes based on what users have ordered in the past, making it easier to find something delicious.

Objectives for a Food App

1. **User Features**:
    - Browse restaurants, view menus, and place orders.

- Secure user authentication with real-time order tracking.

2. **Restaurant Management**:
   - Admin dashboard for restaurant owners to manage menus and orders.
   - Enable ratings, reviews, and user feedback integration.
3. **Payment and Support**:
   - Seamless payment processing with multiple options.
   - Integrated customer support via chat or FAQs.
4. **Technical Goals**:
   - Scalable backend with Node.js/Express and MongoDB.
   - Responsive, dynamic frontend with React.

3.4 Feasibility Analysis

1. Technical Feasibility

Technology Stack: Use of modern frameworks like MERN (MongoDB, Express.js, React.js, Node.js) for scalability and efficiency.

Key Features:

- ✓ User-friendly interface for menu browsing and order placement.
- ✓ Real-time order tracking using geolocation.
- ✓ Secure payment integration (e.g., Stripe, PayPal).
- ✓ Notifications for order status updates.

Challenges:

- ✓ Ensuring app scalability to handle peak traffic.
- ✓ Implementing robust cybersecurity to protect user data.

2. Economical feasibility

- The cost to conduct a full system investigation.
- The cost of hardware and software for the class of application being considered.
- The benefits in the form of reduced cost.

3. Financial Feasibility

- **Initial Costs:**
- App development (design, coding, testing).
- Marketing and user acquisition campaigns.
- Infrastructure setup (servers, hosting).
- **Revenue Streams:**
- Commissions from restaurants.
- Delivery fees or subscription plans.
- In-app advertisements and promotions.
- **Profitability:** Break-even analysis depends on user adoption rates, average order value, and operational efficiency.

CHAPTER 4

PROJECT IMPLEMENTATION

4.1 Methodology

Methodology for enhancing for Web Development efficiency using the MERN stack.

Firstly we have learnt the technologies that were required in order to complete the project.

That is HTML, CSS, java script, React, Tailwind CSS for the front end, working with Mongo DB for Backend and also use Node Js. The order in which we have learnt the technology is:

1) Concepts of HTML,CSS, Bootstrap
2) Concepts of Java script
3) Concepts of Node Js and React
4) Working with Tailwind CSS
5) Working with Mongo DB

HTML:

stands for **Hyper Text Markup Language**. It is the standard markup language used to create web pages. HTML is a combination of Hypertext and Markup language. Hypertext defines the link between web pages. A markup language is used to define the text document within the tag to define the structure of web pages.

This language is used to annotate (make notes for the computer) text so that a machine can understand it and manipulate text accordingly. Most markup languages (e.g. HTML) are human readable. The language uses tags to define what manipulation has to be done on the text.

Javascript:

JavaScript is a lightweight, cross-platform, single-threaded, and interpreted compiled programming language. It is also known as the scripting language for web pages. It is well-known for the development of web pages, and many non-browser environments also use it.

JavaScript is a weakly typed language (dynamically typed). JavaScript can be used for Client side developments as well as Server-side developments. JavaScript is both an imperative and declarative type of language. JavaScript contains a standard library of objects, like **Array, Date, and Math,** and a core set of language elements like **operators**, **control structures**, and **statements**.

Figure 4.1 Difference between HTML, CSS and Javascript

NodeJS:

Node.js (Node) is an Open Source, cross-platform runtime environment for executing JavaScript code. Node is used extensively for server-side programming, making it possible for developers to use JavaScript for client-side and server-side code without needing to learn an

additional language. Node is sometimes referred to as a programming language or software development framework, but neither is true; it is strictly a JavaScript runtime.

REACT.JS:

React JS is a declarative, efficient, and flexible JavaScript library for building reusable UI components. It is an open-source, component-based front end library responsible only for the view layer of the application. It was created by **Jordan Walke,** who was a software engineer at **Facebook.** It was initially developed and maintained by Facebook and was later used in its products like **WhatsApp** & **Instagram.** Facebook developed ReactJS in **2011** in its newsfeed section, but it was released to the public in the month of **May 2013.**

Today, most of the websites are built using MVC (model view controller) architecture. In MVC architecture, React is the 'V' which stands for view, whereas the architecture is provided by the Redux or Flux.

A React JS application is made up of multiple components, each component responsible for outputting a small, reusable piece of HTML code. The components are the heart of all React applications. These Components can be nested with other components to allow complex applications to be built of simple building blocks. React JS uses virtual DOM based mechanism to fill data in HTML DOM. The virtual DOM works fast as it only changes individual DOM elements instead of reloading complete DOM every time.

To create React app, we write React components that correspond to various elements. We organize these components inside higher level components which define the application structure. For example, we take a form that consists of many elements like input fields, labels, or buttons. We can write each element of the form as React components, and then we combine it into a higher level component, i.e., the form component itself. The form components would specify the structure of the form along with elements inside of it.

Figure 4.2 React and NodeJS

Tailwind CSS:

Tailwind CSS is an open-source framework used to style your website in HTML without external CSS. It is a utility-first framework used for building custom user interfaces. It helps us build a website that stands out from the rest. You can style your website using the pre-defined utility classes without external CSS. Tailwind CSS is not a UI kit and doesn't have a default kit or UI components. Let's learn how to install Tailwind CSS on your machine now.

Figure 4.3 Tailwindcss

Mongo-DB:

Mongo DB, the most popular No SQL database, is an open-source document-oriented database.

The term ‘No SQL’ means ‘non-relational’. It means that Mongo DB isn’t based on the table-like relational database structure but provides an altogether different mechanism for storage and retrieval of data. This format of storage is called BSON (similar to JSON format).

Features of Mongo-DB:

- **Document Oriented:** Mongo DB stores the main subject in the minimal number of documents and not by breaking it up into multiple relational structures like RDBMS. For example, it stores all the information of a computer in a single document called Computer and not in distinct relational structures like CPU, RAM, Hard disk, etc.
- **Indexing:** Without indexing, a database would have to scan every document of a collection to select those that match the query which would be inefficient. So, for efficient searching Indexing is a must and Mongo DB uses it to process huge volumes of data in very less time.
- **Scalability:** Mongo DB scales horizontally using shading (partitioning data across various servers). Data is partitioned into data chunks using the shard key, and these data chunks are evenly distributed across shards that reside across many physical servers.

  Also, new machines can be added to a running database.
- **Replication and High Availability:** Mongo DB increases the data availability with multiple copies of data on different servers. By providing redundancy, it protects the database from hardware failures. If one server goes down, the data can be retrieved easily from other active servers which also had the data stored on them.
- **Aggregation:** Aggregation operations process data records and return the computed results. It is similar to the GROUPBY clause in SQL. A few aggregation expressions are sum, avg, min, max, etc .

Figure 4.4 MongoDB

Redux Toolkit:

Redux Toolkit (RTK) is the official, recommended way to use Redux for managing state in modern JavaScript applications. It simplifies the process of writing Redux logic by providing powerful abstractions and built-in tools to reduce boilerplate code and improve developer experience

Key Features of Redux Toolkit

1. **Simplified Setup:**
- RTK provides a `configureStore` method that automatically sets up the Redux store with sensible defaults like middleware, the Redux DevTools integration, and a default state setup.
2. **Slices for Modularity:**
- RTK introduces "slices" via the `createSlice` function, which combines actions and reducers into a single, modular file, making the state management process more intuitive.
3. **Built-in Middleware:**
- Automatically includes middleware like `redux-thunk` for handling asynchronous logic, and allows easy addition of custom middleware.
4. **Immutability with Immer:**

- Built-in use of **Immer.js** allows developers to write "mutating" code for state updates that are converted to immutable updates under the hood.

5. **Asynchronous Logic with** createAsyncThunk**:**

- Provides utilities to handle asynchronous actions, simplifying API calls and loading state management.

6. **Boilerplate Reduction:**

- Eliminates repetitive patterns like action type strings, action creators, and switch cases in reducers.

7. **Developer Tooling:**

- Seamless integration with Redux DevTools for debugging and time-travel debugging.

8. **TypeScript Support:**

- Excellent TypeScript support with built-in type inference, making it easier to use in strongly typed applications.

Figure 4.5 Redux Toolkit

4.2 SOFTWARES USED

1) Visual Studio Code

Visual Studio Code is a lightweight but powerful source code editor which runs on your desktop and is available for Windows, Mac OS and Linux. It comes with built-in support for JavaScript, Type Script and Node.js and has a rich ecosystem of extensions for other languages and runtimes (such as C++, C#, Java, Python, PHP, Go, .NET).

Figure 4.6 Visual Studio Code

Mongo DB

Mongo DB is an open source No SQL database management program. No SQL (Not only SQL) is used as an alternative to traditional relational databases. No SQL databases are quite useful for working with large sets of distributed data. Mongo DB is a tool that can manage document oriented information, store or retrieve information.

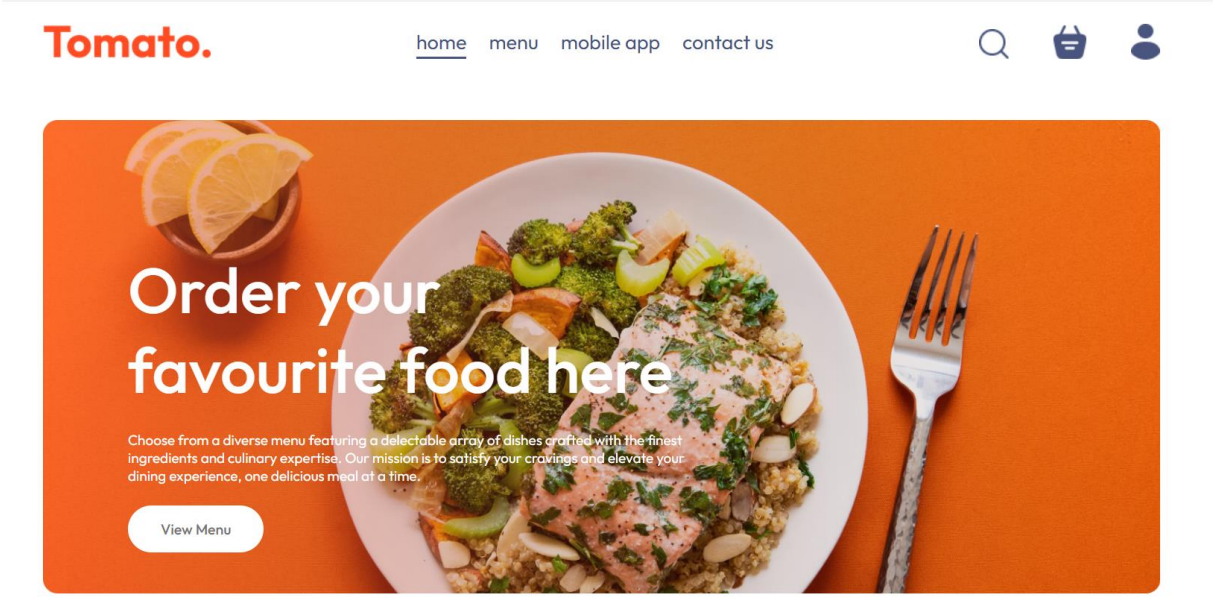
Mongo DB is used for high-volume data storage, helping organizations store large amounts of data while still performing rapidly. Organizations also use Mongo DB for its ad-hoc queries, indexing, load balancing, aggregation, server-side JavaScript execution and other features.

Figure 4.7 MongoDB

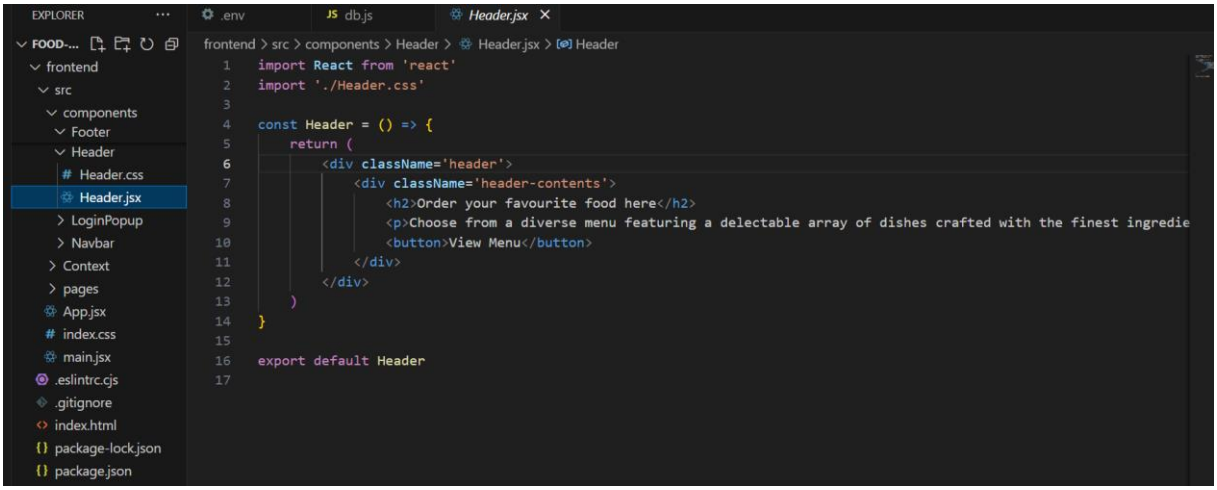
CHAPTER 5
RESULT AND DISCUSSION

5.1 Screenshots

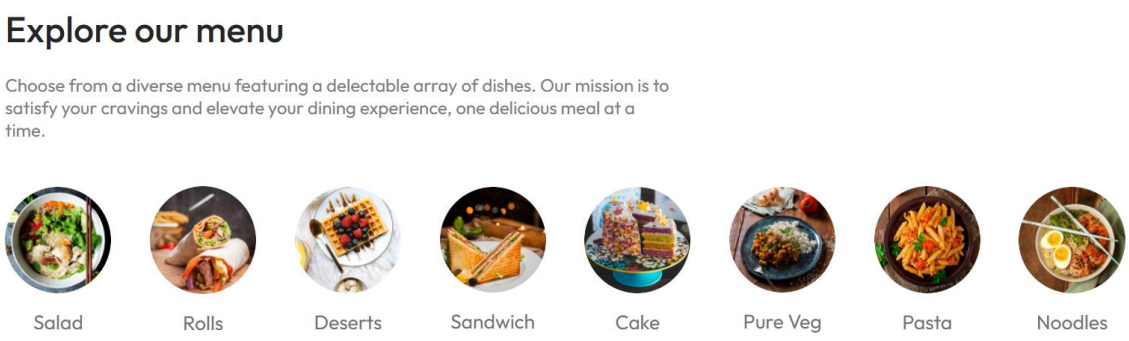
Header Component: The header is a critical part of the app that provides navigation across different sections, such as the home page, the explore menu, and possibly a cart or contact page.



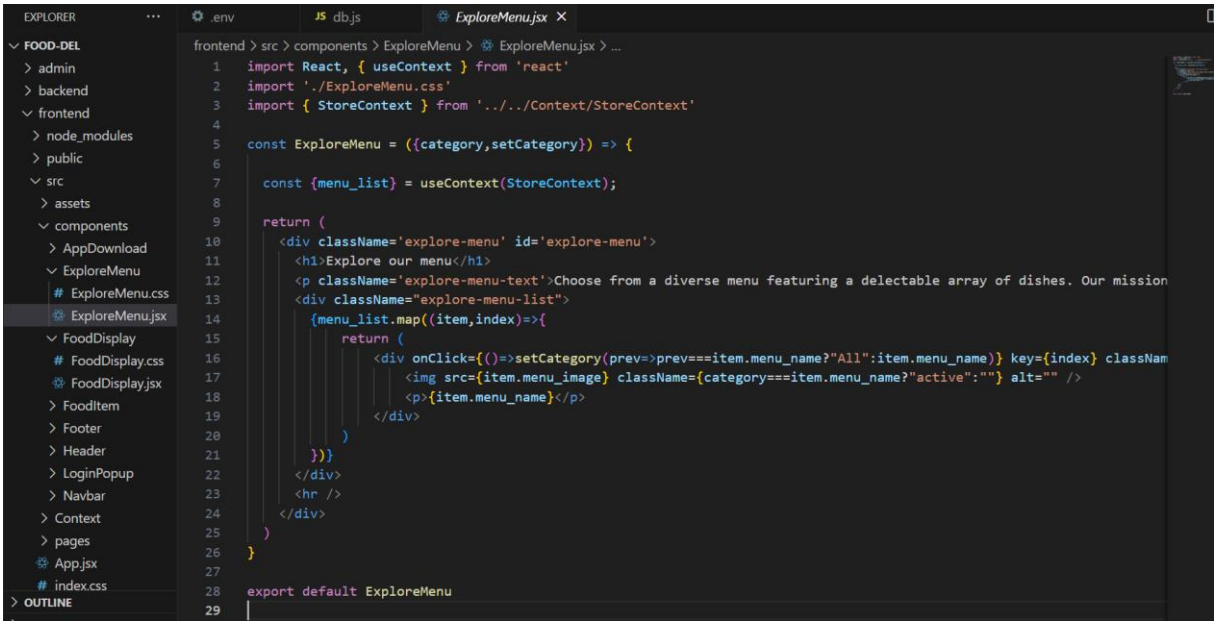
Code:



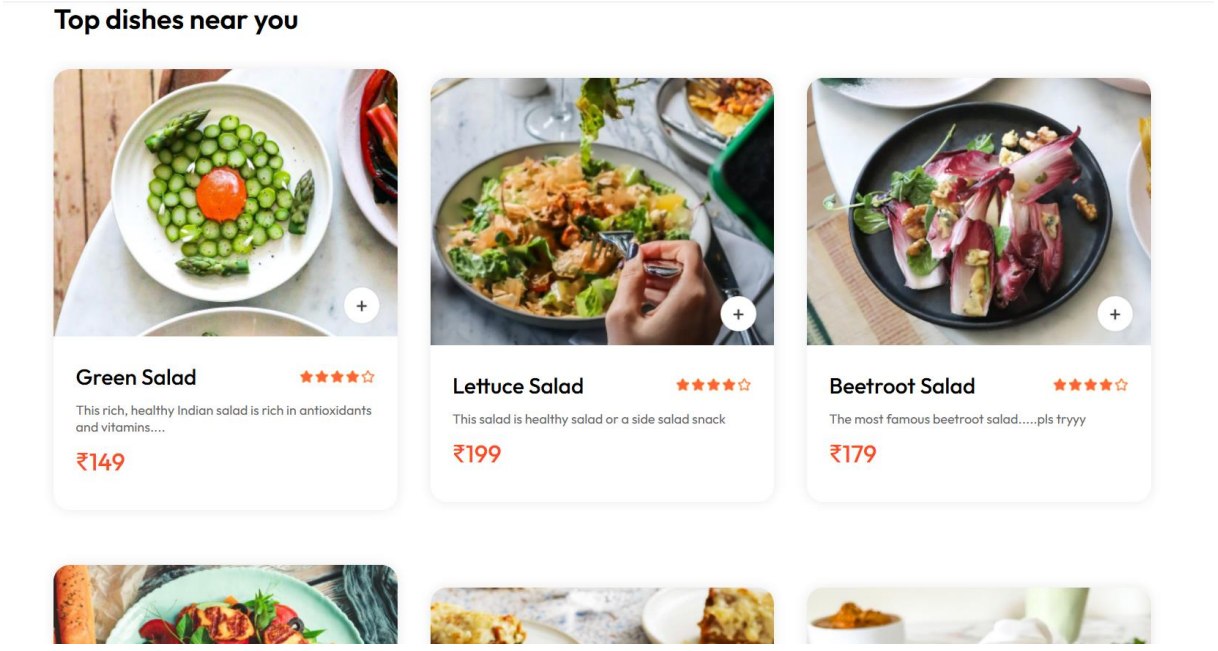
Explore Menu Component: This component lets users explore the different food categories or types of cuisine available in the app. It helps users discover what's on the menu.



Code:



Food Items Component: This component displays the actual food items that users can view, select, or order. Each food item typically has details such as a name, price, and an image.



Burrata and Tomato Salad ★★★★☆

Everything is better with Burrata, especially the tomato salad...

₹229

Lasagna Rolls ★★★★☆

This is snack that you will love to have, any time....

₹99

Mexican Rolls ★★★★☆

best roll ever ...this is soo yummy!!

₹159

Chicken Rolls ★★★★☆

Chicken dinner is a winner....

₹239

Veg Rolls ★★★★☆

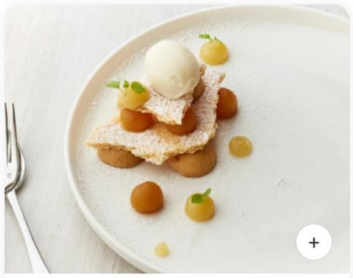
Veg Foodies welcome here.....!

₹199

Strawberry Icecream ★★★★☆

A frozen treat made from fruit...

₹189



Frozen Gelato ice cream ★★★★☆

Unlock the door to Icecream Heaven: a taste sensation....

Angel Strawberry Dessert ★★★★☆

This angel dessert is a wonderful treat when fresh strawberries are readily available....

Vanilla Sizzling Icecream ★★★★☆

Dare to be Different: Discover Icecream bliss....

Frozen Gelato ice cream ★★★★☆

Unlock the door to Icecream Heaven: a taste sensation....

₹249

Angel Strawberry Dessert ★★★★☆

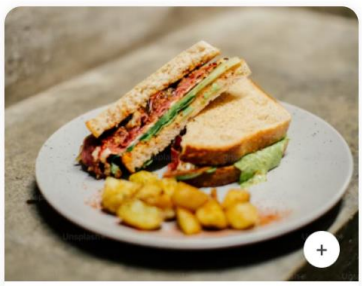
This angel dessert is a wonderful treat when fresh strawberries are readily available....

₹319

Vanilla Sizzling Icecream ★★★★☆

Dare to be Different: Discover Icecream bliss....

₹179



American Sandwich ★★★★☆

"Sandwiches are wonderful. You don't need a spoon or a plate!"

₹169

Double Cheese Sandwich ★★★★☆

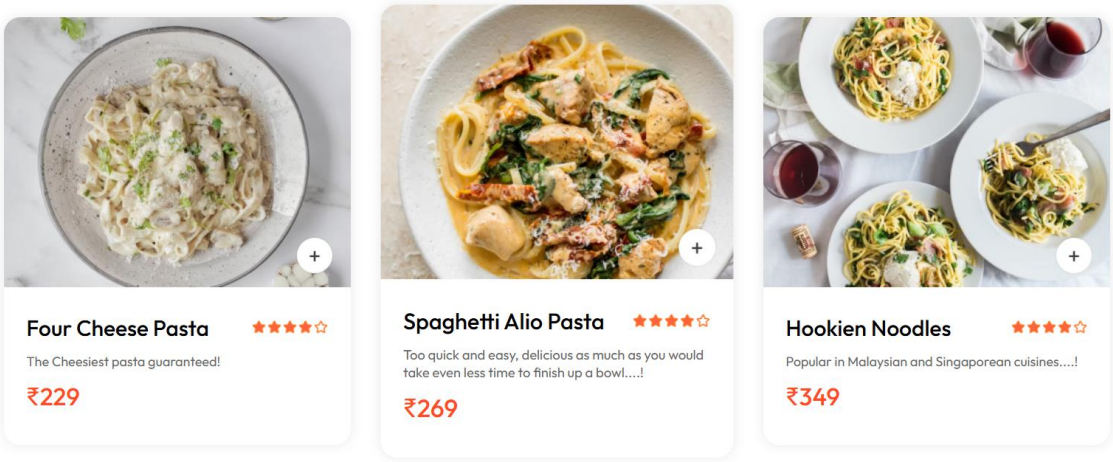
"A sandwich is a meal between two slices of bread."

₹299

Grilled Cheese Sandwich ★★★★☆

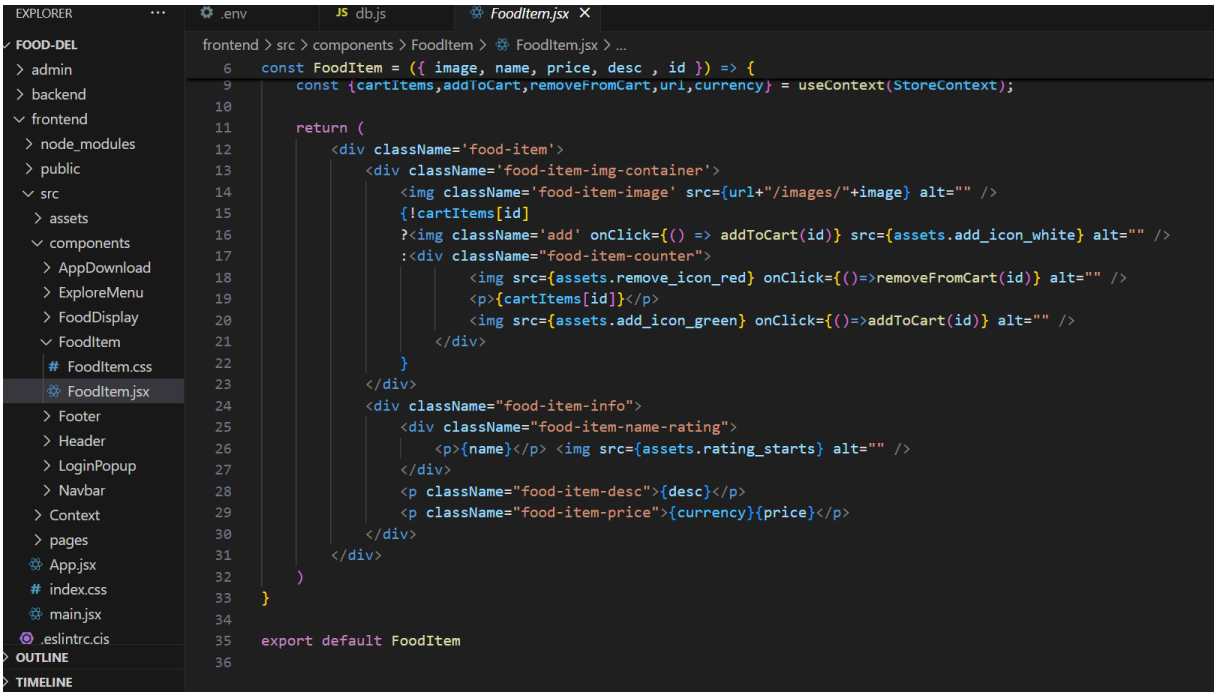
"Life is too short for a bad sandwich."

₹269

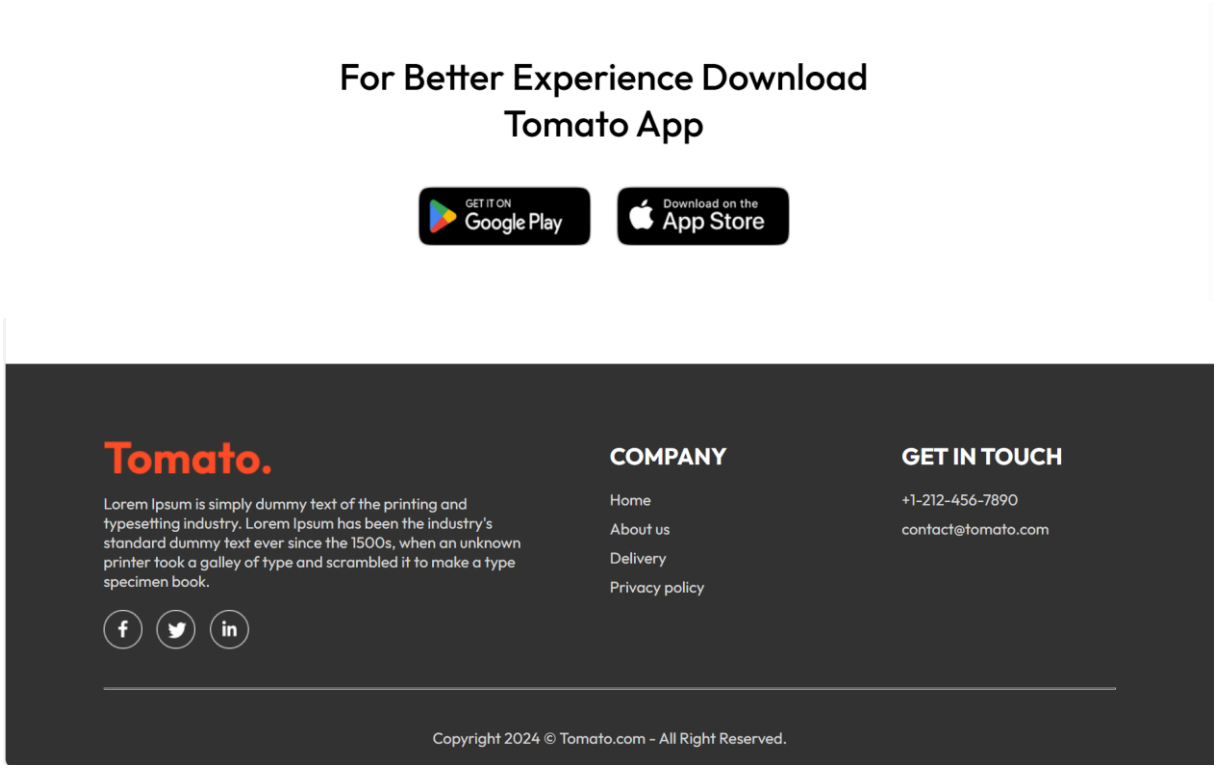


Code:

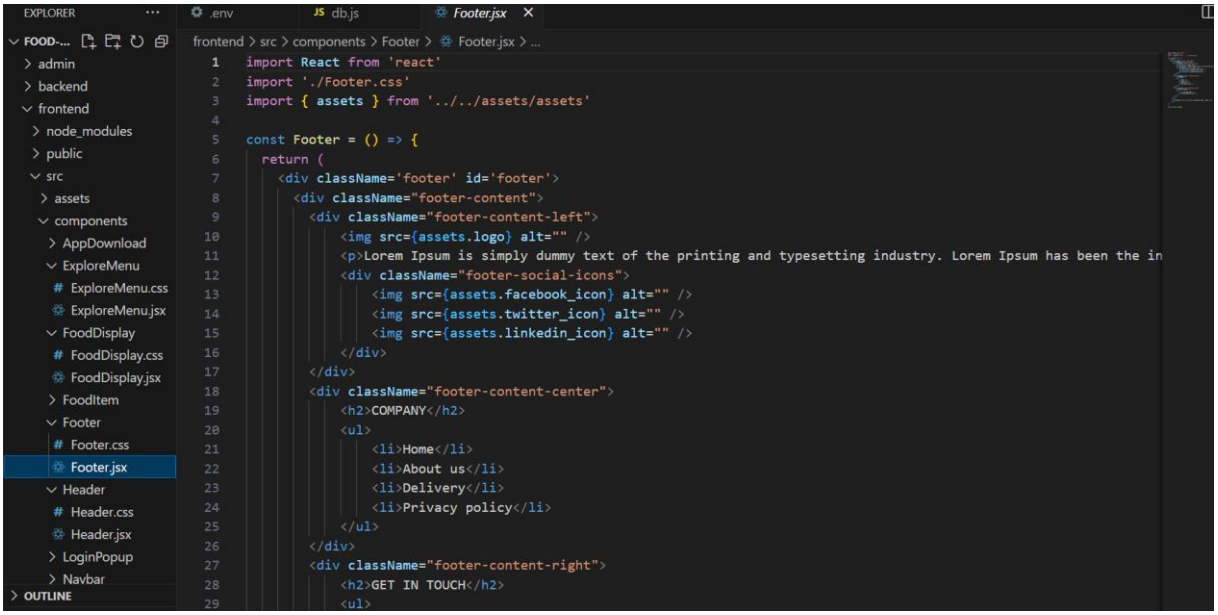
```
EXPLORER                                  .env   db.js   FoodItem.jsx
FOOD-DEL
  admin
  backend
  frontend
    node_modules
    public
    src
      assets
      components
        AppDownload
        ExploreMenu
        FoodDisplay
        FoodItem
          FoodItem.css
          FoodItem.jsx
        Footer
        Header
        LoginPopup
        Navbar
      Context
      pages
    App.jsx
    index.css
    main.jsx
    .eslintrc.cjs
OUTLINE
TIMELINE

frontend > src > components > FoodItem > FoodItem.jsx > ...
1  import React, { useContext, useState } from 'react'
2  import './FoodItem.css'
3  import { assets } from '../../assets/assets'
4  import { StoreContext } from '../../Context/StoreContext';
5
6  const FoodItem = ({ image, name, price, desc , id }) => {
7
8      const [itemCount, setItemCount] = useState(0);
9      const {cartItems,addToCart,removeFromCart,url,currency} = useContext(StoreContext);
10
11     return (
12         <div className='food-item'>
13             <div className='food-item-img-container'>
14                 <img className='food-item-image' src={url+"/images/"+image} alt="" />
15                 {!cartItems[id]
16                 ?<img className='add' onClick={() => addToCart(id)} src={assets.add_icon_white} alt="" />
17                 :<div className="food-item-counter">
18                         <img src={assets.remove_icon_red} onClick={()=>removeFromCart(id)} alt="" />
19                         <p>{cartItems[id]}</p>
20                         <img src={assets.add_icon_green} onClick={()=>addToCart(id)} alt="" />
21                     </div>
22                 }
23             </div>
24             <div className="food-item-info">
25                 <div className="food-item-name-rating">
26                     <p>{name}</p> <img src={assets.rating_starts} alt="" />
27                 </div>
28                 <p className="food-item-desc">{desc}</p>
29                 <p className="food-item-price">{currency}{price}</p>
30             </div>
```

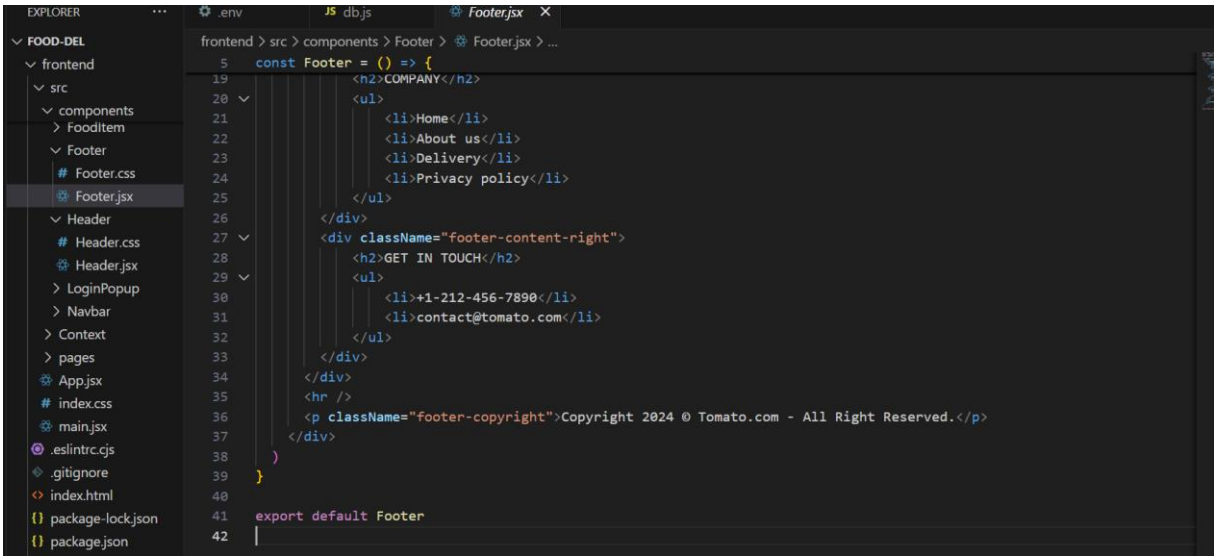


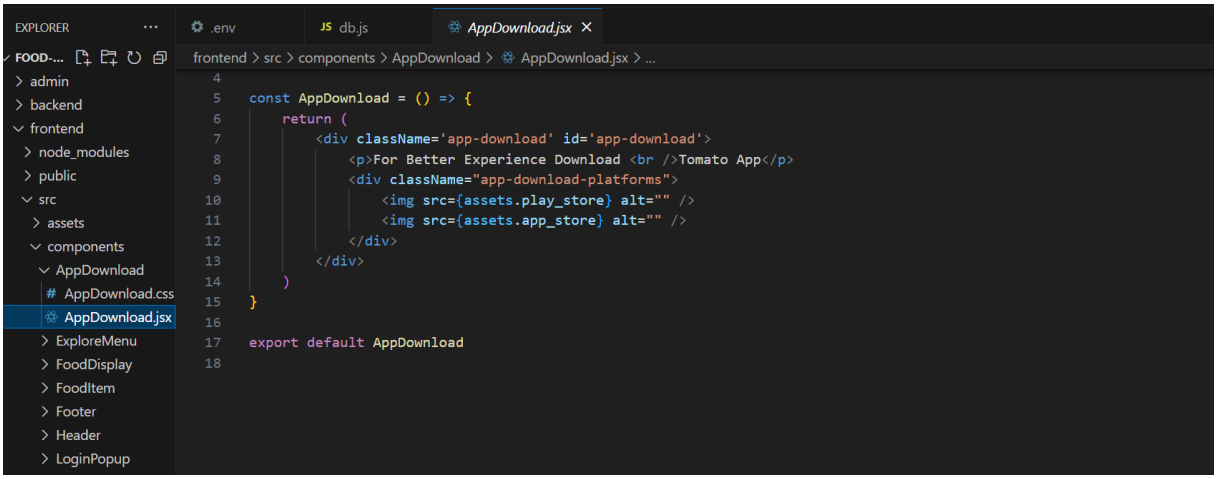
Footer Component: The footer is typically found at the bottom of every page in your app, containing important information or links.



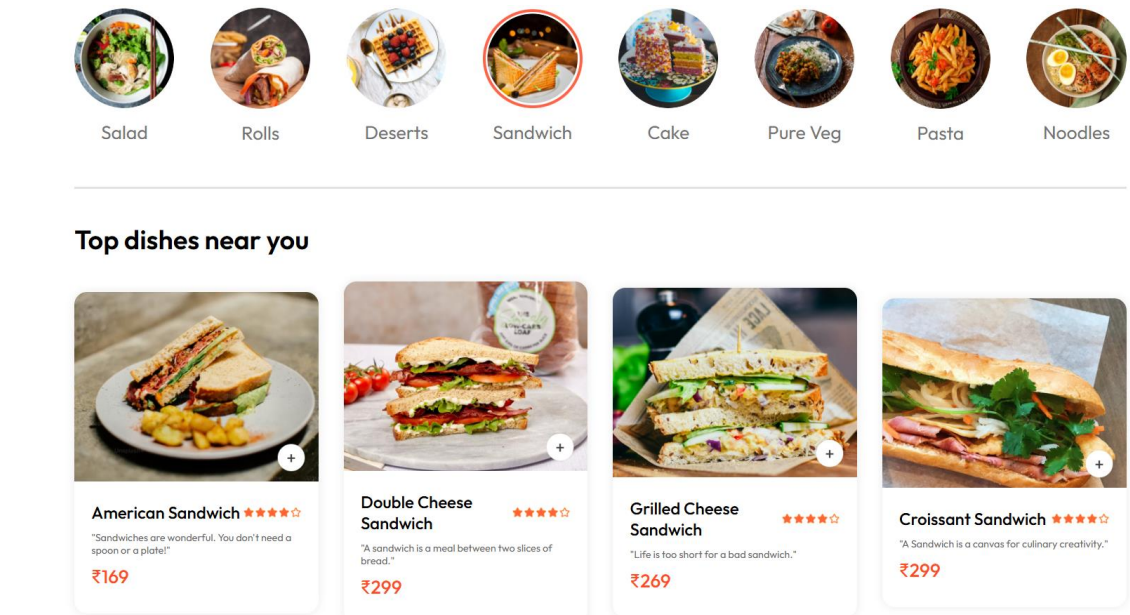
Code:

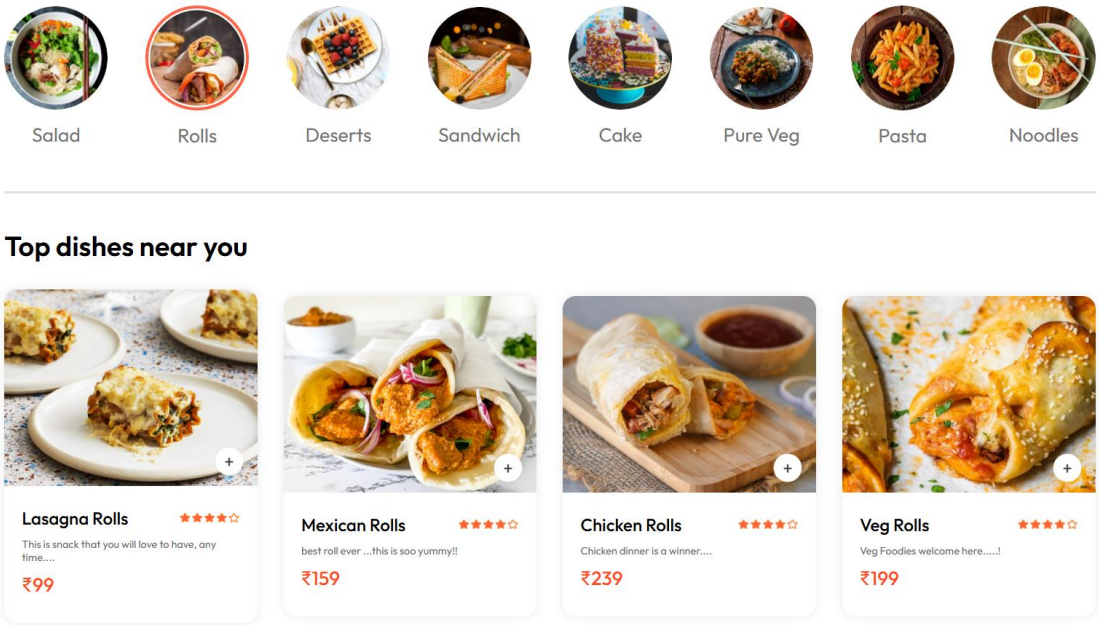

EXPLORER
.env
db.js
Footer.jsx
frontend > src > components > Footer > Footer.jsx > ...
FOOD-...
admin
backend
frontend
node_modules
public
src
assets
components
AppDownload
ExploreMenu
ExploreMenu.css
ExploreMenu.jsx
FoodDisplay
FoodDisplay.css
FoodDisplay.jsx
FoodItem
Footer
Footer.css
Footer.jsx
Header
Header.css
Header.jsx
LoginPopup
Navbar
OUTLINE
1 import React from 'react'
2 import './Footer.css'
3 import { assets } from '../../assets/assets'
4
5 const Footer = () => {
6 return (
7
8
9
10
11 Lorem Ipsum is simply dummy text of the printing and typesetting industry. Lorem Ipsum has been the in
12
13
14
15
16
17
18
19 <h2>COMPANY</h2>
20
21 Home
22 About us
23 Delivery
24 Privacy policy
25
26
27
28 <h2>GET IN TOUCH</h2>
29


EXPLORER
.env
db.js
Footer.jsx
frontend > src > components > Footer > Footer.jsx > ...
FOOD-DEL
frontend
src
components
FoodItem
Footer
Footer.css
Footer.jsx
Header
Header.css
Header.jsx
LoginPopup
Navbar
Context
pages
App.jsx
index.css
main.jsx
.eslintrc.cjs
.gitignore
index.html
package-lock.json
package.json
5 const Footer = () => {
19 <h2>COMPANY</h2>
20
21 Home
22 About us
23 Delivery
24 Privacy policy
25
26
27
28 <h2>GET IN TOUCH</h2>
29
30 +1-212-456-7890
31 contact@tomato.com
32
33
34
35 <hr />
36 Copyright 2024 © Tomato.com - All Right Reserved.
37
38)
39 }
40
41 export default Footer
42

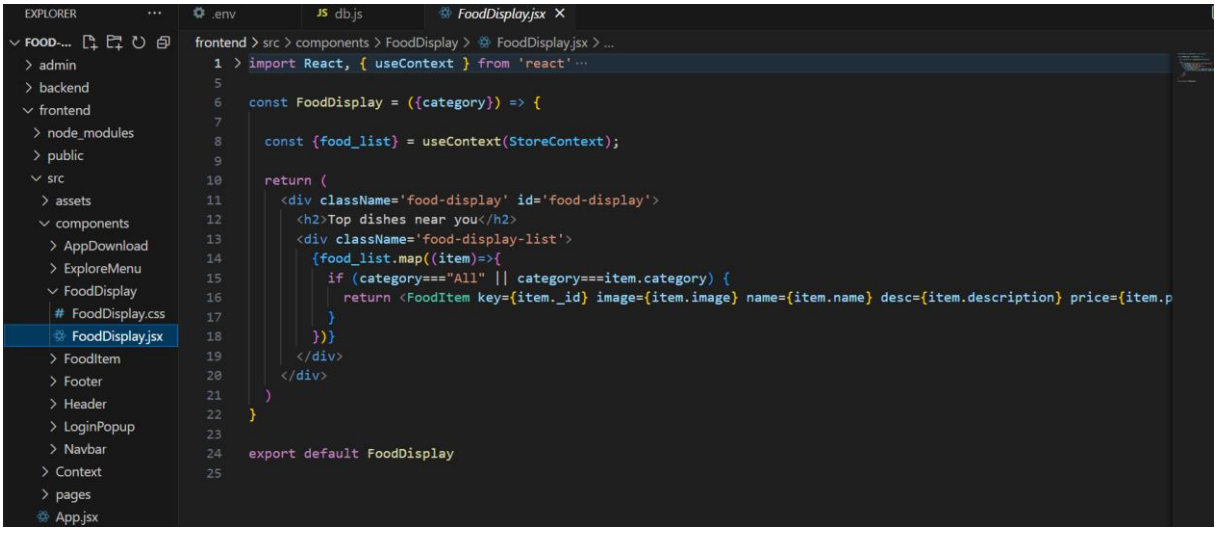


Food display:

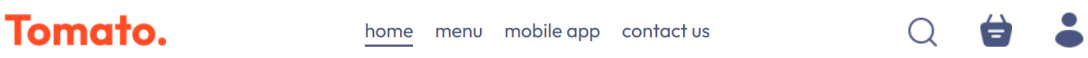




Code:

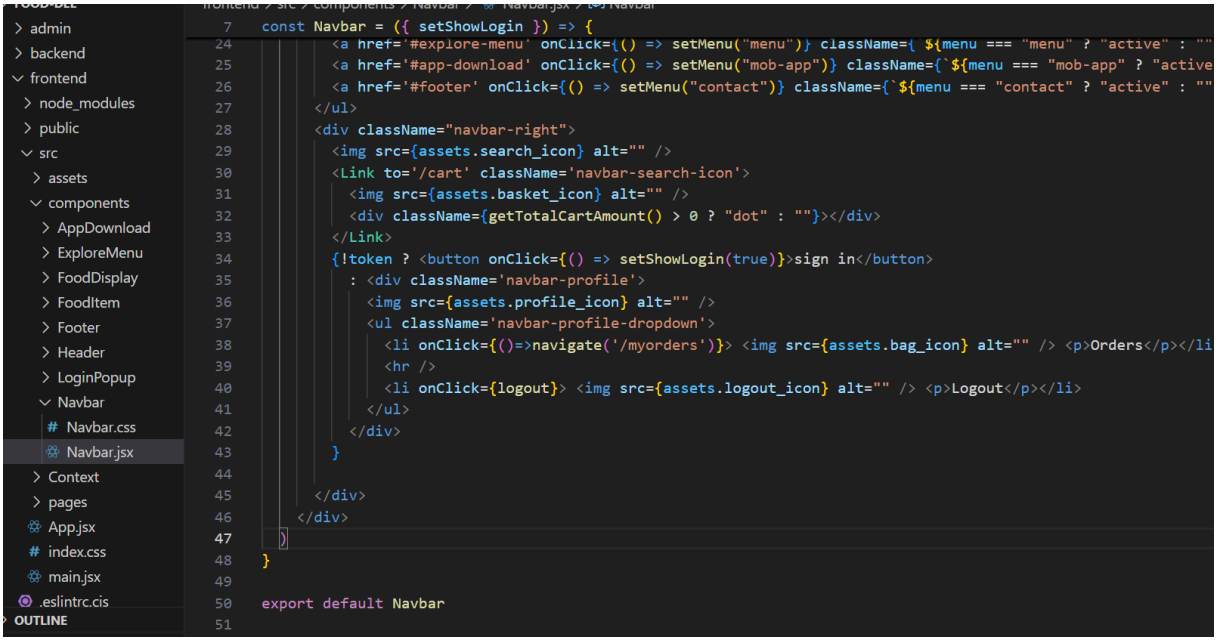


Navbar component :

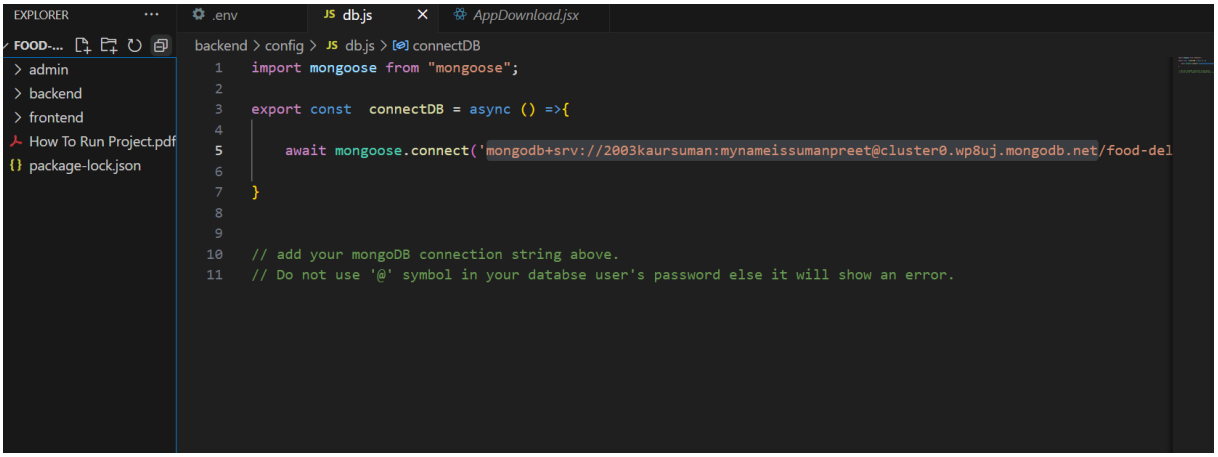


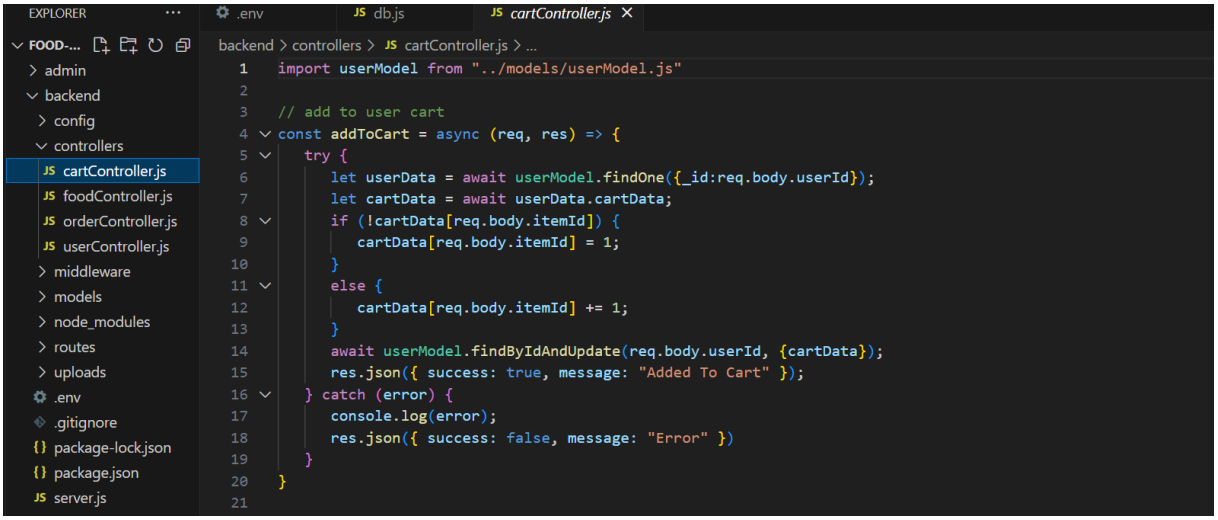
Code:


EXPLORER
.env
db.js
Navbar.jsx
frontend > src > components > Navbar > Navbar.jsx > ...
admin
backend
frontend
node_modules
public
src
assets
components
AppDownload
ExploreMenu
FoodDisplay
FoodItem
Footer
Header
LoginPopup
Navbar
Navbar.css
Navbar.jsx
Context
pages
App.jsx
index.css
main.jsx
.eslintrc.cis
OUTLINE
1 import React, { useContext, useEffect, useState } from 'react'
2 import './Navbar.css'
3 import { assets } from '../../assets/assets'
4 import { Link, useNavigate } from 'react-router-dom'
5 import { StoreContext } from '../../Context/StoreContext'
6
7 const Navbar = ({ setShowLogin }) => {
8
9 const [menu, setMenu] = useState("home");
10 const { getTotalCartAmount, token ,setToken } = useContext(StoreContext);
11 const navigate = useNavigate();
12
13 const logout = () => {
14 localStorage.removeItem("token");
15 setToken("");
16 navigate('/')
17 }
18
19 return (
20
21 <Link to='/'></Link>
22 <ul className="navbar-menu">
23 <Link to="/" onClick={() => setMenu("home")} className={`${menu === "home" ? "active" : ""}`}>home</Link>
24 <a href='#explore-menu' onClick={() => setMenu("menu")} className={`${menu === "menu" ? "active" : ""}`}>menu
25 <a href='#app-download' onClick={() => setMenu("mob-app")} className={`${menu === "mob-app" ? "active" : ""}`
26 <a href='#footer' onClick={() => setMenu("contact")} className={`${menu === "contact" ? "active" : ""}`}>cont
27
28

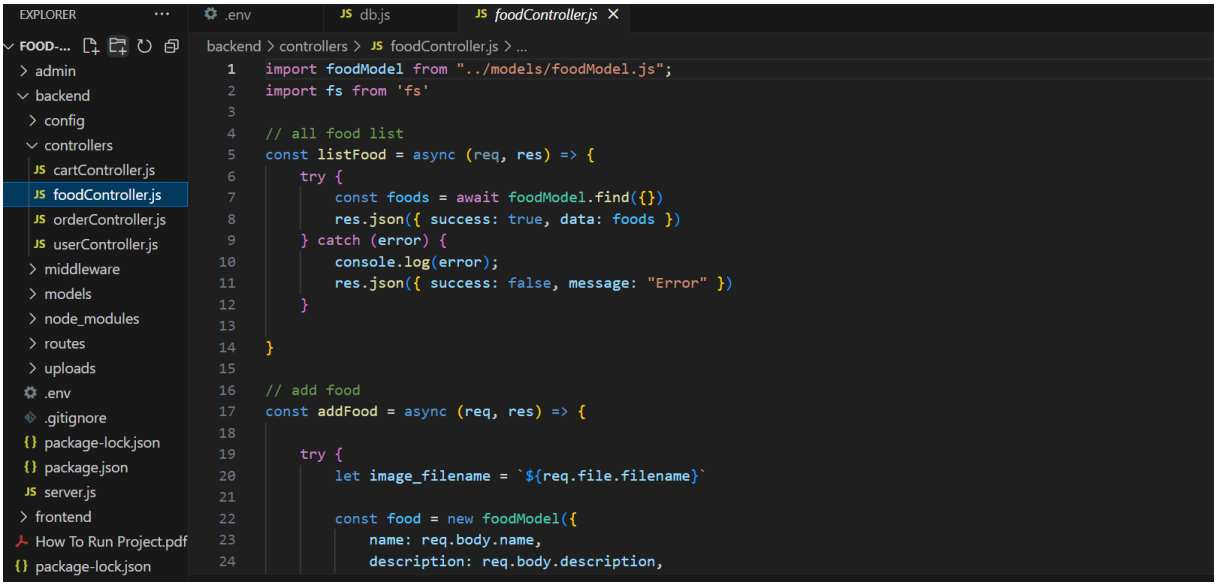

admin
backend
frontend
node_modules
public
src
assets
components
AppDownload
ExploreMenu
FoodDisplay
FoodItem
Footer
Header
LoginPopup
Navbar
Navbar.css
Navbar.jsx
Context
pages
App.jsx
index.css
main.jsx
.eslintrc.cis
OUTLINE
7 const Navbar = ({ setShowLogin }) => {
24 <a href='#explore-menu' onClick={() => setMenu("menu")} className={ ${menu === "menu" ? "active" : ""
25 <a href='#app-download' onClick={() => setMenu("mob-app")} className={`${menu === "mob-app" ? "active
26 <a href='#footer' onClick={() => setMenu("contact")} className={`${menu === "contact" ? "active" : ""
27
28
29
30 <Link to='/cart' className='navbar-search-icon'>
31
32 0 ? "dot" : ""}>
33 </Link>
34 {!token ? <button onClick={() => setShowLogin(true)}>sign in</button>
35 :
36
37 <ul className='navbar-profile-dropdown'>
38 <li onClick={()=>navigate('/myorders')}> Orders </li
39 <hr />
40 <li onClick={logout}> Logout
41
42
43 }
44
45
46
47)
48 }
49
50 export default Navbar
51

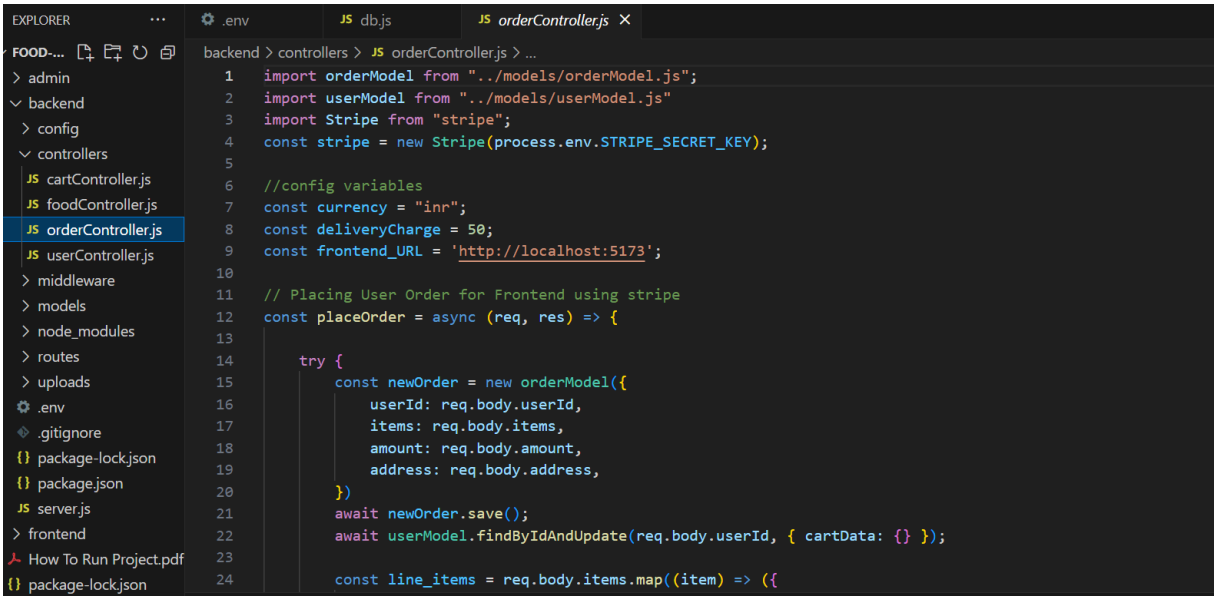
Backend:


EXPLORER
.env
db.js
AppDownload.jsx
FOOD-...
backend > config > db.js > connectDB
admin
backend
frontend
How To Run Project.pdf
package-lock.json
1 import mongoose from "mongoose";
2
3 export const connectDB = async () =>{
4
5 await mongoose.connect('mongodb+srv://2003kaursuman:mynameissumanpreet@cluster0.wp8uj.mongodb.net/food-del
6
7 }
8
9
10 // add your mongoDB connection string above.
11 // Do not use '@' symbol in your databse user's password else it will show an error.

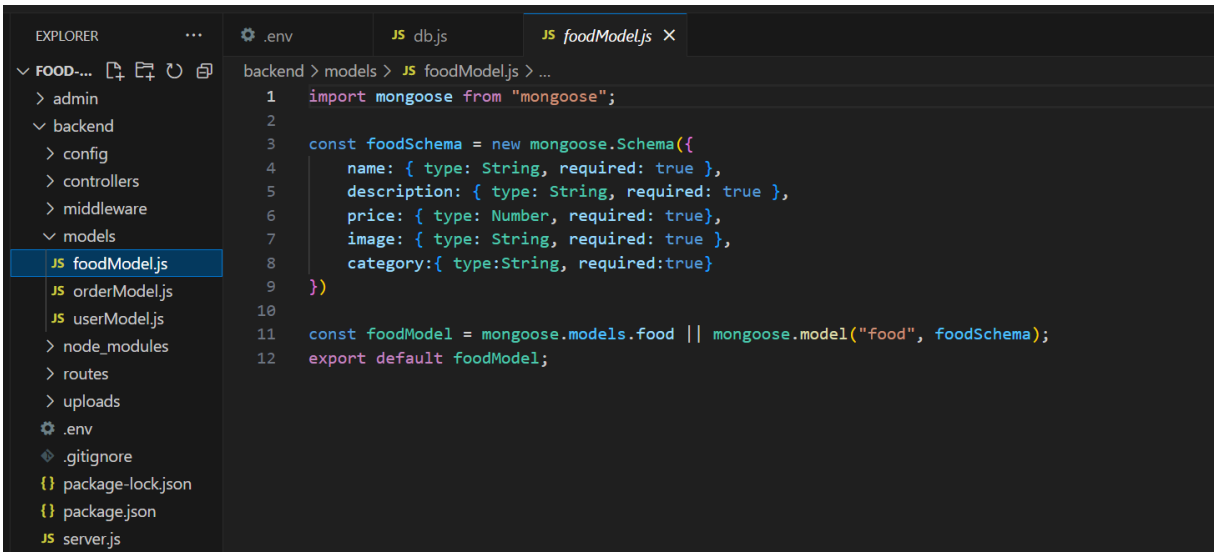

EXPLORER
.env
db.js
cartController.js
FOOD-...
backend > controllers > cartController.js > ...
admin
backend
config
controllers
cartController.js
foodController.js
orderController.js
userController.js
middleware
models
node_modules
routes
uploads
.env
.gitignore
package-lock.json
package.json
server.js
1 import userModel from "../models/userModel.js"
2
3 // add to user cart
4 const addToCart = async (req, res) => {
5 try {
6 let userData = await userModel.findOne({_id:req.body.userId});
7 let cartData = await userData.cartData;
8 if (!cartData[req.body.itemId]) {
9 cartData[req.body.itemId] = 1;
10 }
11 else {
12 cartData[req.body.itemId] += 1;
13 }
14 await userModel.findByIdAndUpdate(req.body.userId, {cartData});
15 res.json({ success: true, message: "Added To Cart" });
16 } catch (error) {
17 console.log(error);
18 res.json({ success: false, message: "Error" })
19 }
20 }
21

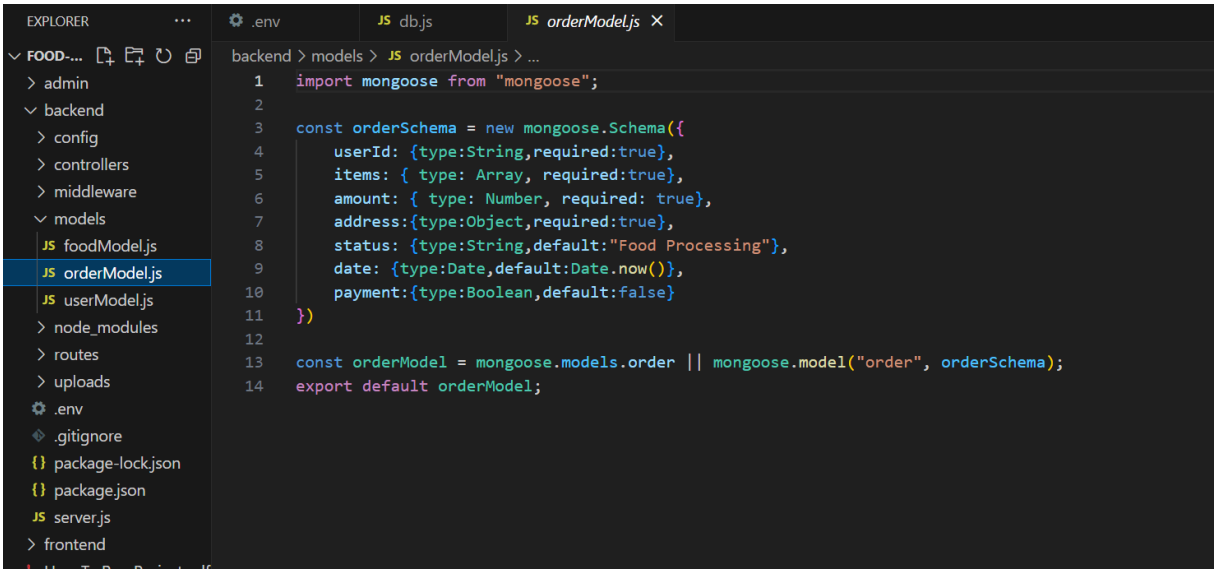

EXPLORER
.env
db.js
cartController.js
backend > controllers > cartController.js > getCart
admin
backend
config
controllers
cartController.js
foodController.js
orderController.js
userController.js
middleware
models
node_modules
routes
uploads
.env
.gitignore
package-lock.json
package.json
server.js
frontend
How To Run Project.pdf
// remove food from user cart
const removeFromCart = async (req, res) => {
try {
let userData = await userModel.findById(req.body.userId);
let cartData = await userData.cartData;
if (cartData[req.body.itemId] > 0) {
cartData[req.body.itemId] -= 1;
}
await userModel.findByIdAndUpdate(req.body.userId, {cartData});
res.json({ success: true, message: "Removed From Cart" });
} catch (error) {
console.log(error);
res.json({ success: false, message: "Error" })
}
}
// get user cart
const getCart = async (req, res) => {
try {
let userData = await userModel.findById(req.body.userId);
let cartData = await userData.cartData;
res.json({ success: true, cartData:cartData });
} catch (error) {

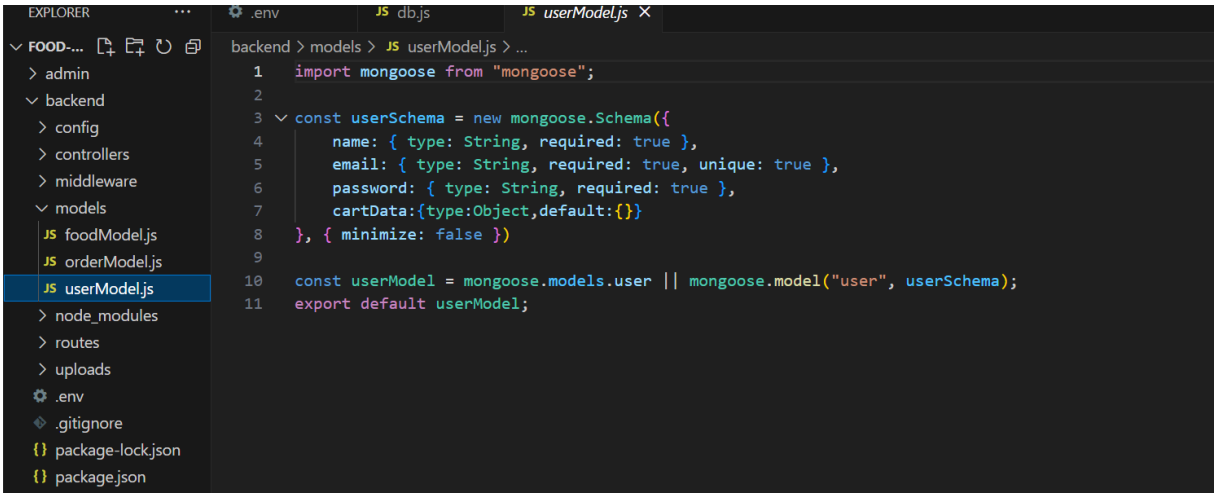

EXPLORER
.env
db.js
foodController.js
backend > controllers > foodController.js > ...
admin
backend
config
controllers
cartController.js
foodController.js
orderController.js
userController.js
middleware
models
node_modules
routes
uploads
.env
.gitignore
package-lock.json
package.json
server.js
frontend
How To Run Project.pdf
import foodModel from "../models/foodModel.js";
import fs from 'fs'
// all food list
const listFood = async (req, res) => {
try {
const foods = await foodModel.find({})
res.json({ success: true, data: foods })
} catch (error) {
console.log(error);
res.json({ success: false, message: "Error" })
}
}
// add food
const addFood = async (req, res) => {
try {
let image_filename = `${req.file.filename}`
const food = new foodModel({
name: req.body.name,
description: req.body.description,


EXPLORER
.env
db.js
orderController.js
backend > controllers > orderController.js > ...
FOOD-...
admin
backend
config
controllers
cartController.js
foodController.js
orderController.js
userController.js
middleware
models
node_modules
routes
uploads
.env
.gitignore
package-lock.json
package.json
server.js
frontend
How To Run Project.pdf
package-lock.json
import orderModel from "../models/orderModel.js";
import userModel from "../models/userModel.js"
import Stripe from "stripe";
const stripe = new Stripe(process.env.STRIPE_SECRET_KEY);
//config variables
const currency = "inr";
const deliveryCharge = 50;
const frontend_URL = 'http://localhost:5173';
// Placing User Order for Frontend using stripe
const placeOrder = async (req, res) => {
try {
const newOrder = new orderModel({
userId: req.body.userId,
items: req.body.items,
amount: req.body.amount,
address: req.body.address,
})
await newOrder.save();
await userModel.findByIdAndUpdate(req.body.userId, { cartData: {} });
const line_items = req.body.items.map((item) => ({

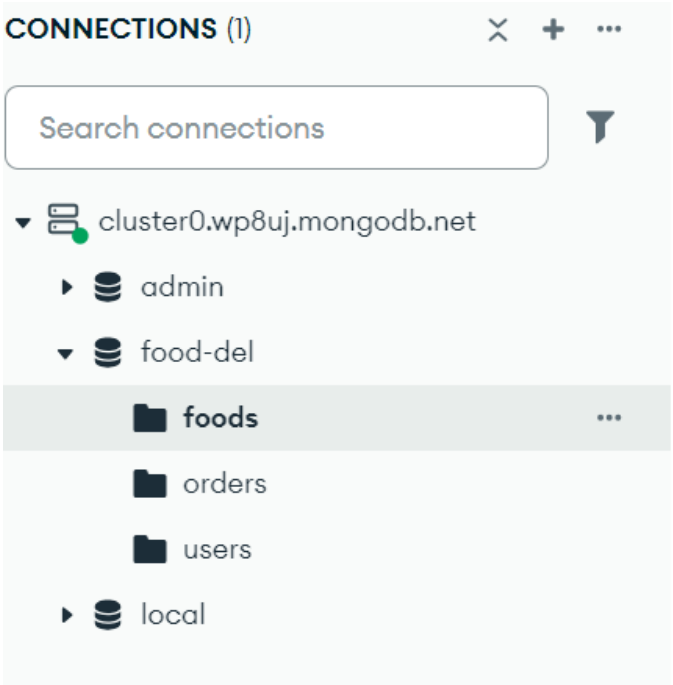

EXPLORER
.env
db.js
userController.js
backend > controllers > userController.js > ...
FOOD-...
admin
backend
config
controllers
cartController.js
foodController.js
orderController.js
userController.js
middleware
models
node_modules
routes
uploads
.env
.gitignore
package-lock.json
package.json
server.js
frontend
How To Run Project.pdf
package-lock.json
import jwt from "jsonwebtoken";
import bcrypt from "bcrypt";
import validator from "validator";
import userModel from "../models/userModel.js";
//create token
const createToken = (id) => {
return jwt.sign({id}, process.env.JWT_SECRET);
}
//login user
const loginUser = async (req,res) => {
const {email, password} = req.body;
try{
const user = await userModel.findOne({email})
if(!user){
return res.json({success:false,message: "User does not exist"})
}
const isMatch = await bcrypt.compare(password, user.password)
if(!isMatch){
return res.json({success:false,message: "Invalid credentials"})

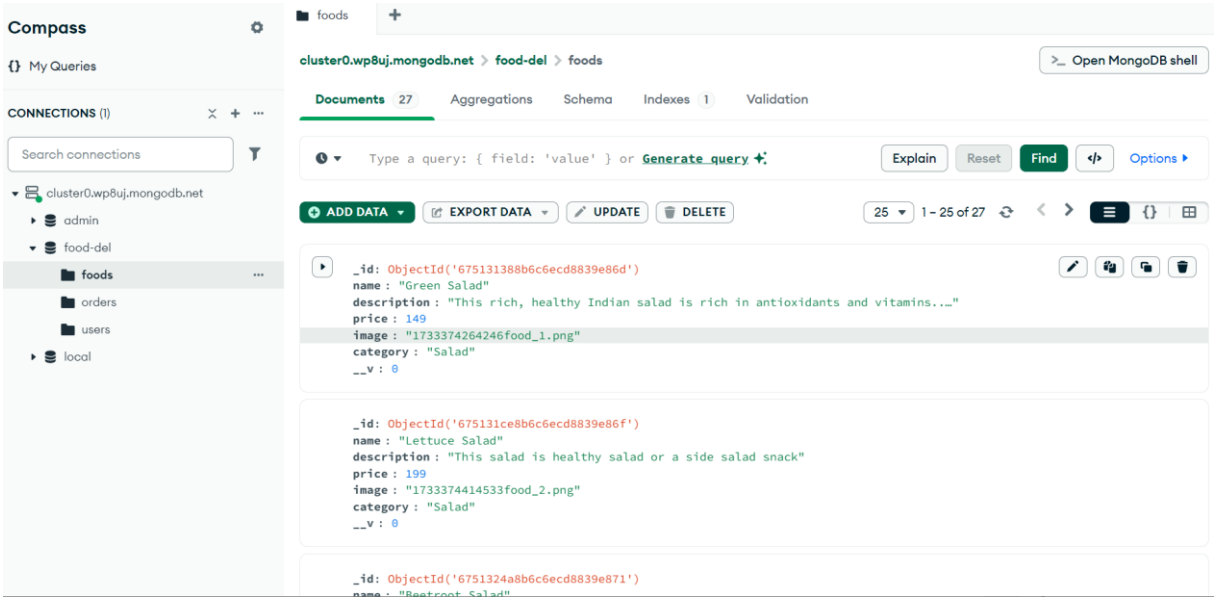

EXPLORER
FOOD-...
admin
backend
config
controllers
middleware
models
foodModel.js
orderModel.js
userModel.js
node_modules
routes
uploads
.env
.gitignore
package-lock.json
package.json
server.js
.env
db.js
foodModel.js
backend > models > foodModel.js > ...
1 import mongoose from "mongoose";
2
3 const foodSchema = new mongoose.Schema({
4 name: { type: String, required: true },
5 description: { type: String, required: true },
6 price: { type: Number, required: true},
7 image: { type: String, required: true },
8 category:{ type:String, required:true}
9 })
10
11 const foodModel = mongoose.models.food || mongoose.model("food", foodSchema);
12 export default foodModel;

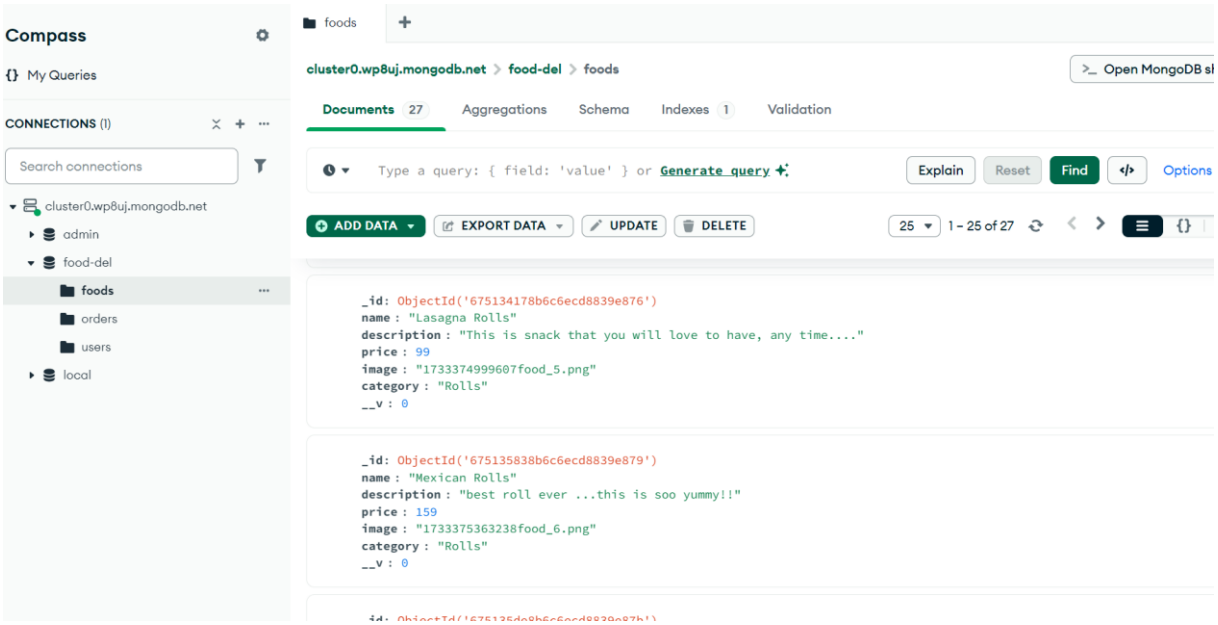

EXPLORER
FOOD-...
admin
backend
config
controllers
middleware
models
foodModel.js
orderModel.js
userModel.js
node_modules
routes
uploads
.env
.gitignore
package-lock.json
package.json
server.js
frontend
.env
db.js
orderModel.js
backend > models > orderModel.js > ...
1 import mongoose from "mongoose";
2
3 const orderSchema = new mongoose.Schema({
4 userId: {type:String,required:true},
5 items: { type: Array, required:true},
6 amount: { type: Number, required: true},
7 address:{type:Object,required:true},
8 status: {type:String,default:"Food Processing"},
9 date: {type:Date,default:Date.now()},
10 payment:{type:Boolean,default:false}
11 })
12
13 const orderModel = mongoose.models.order || mongoose.model("order", orderSchema);
14 export default orderModel;

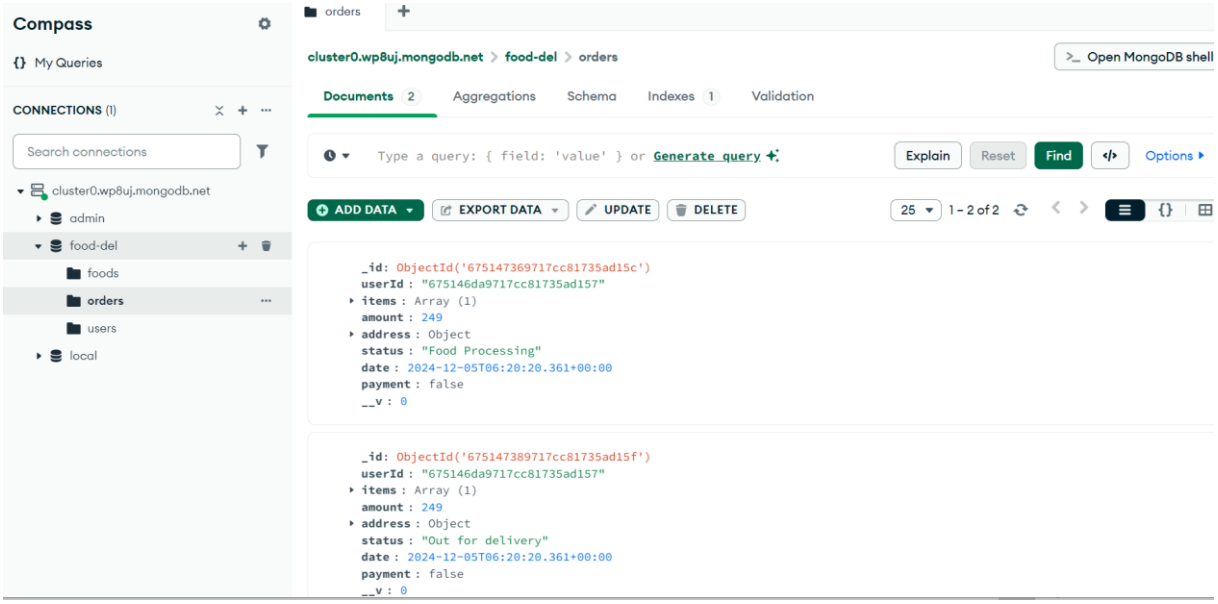


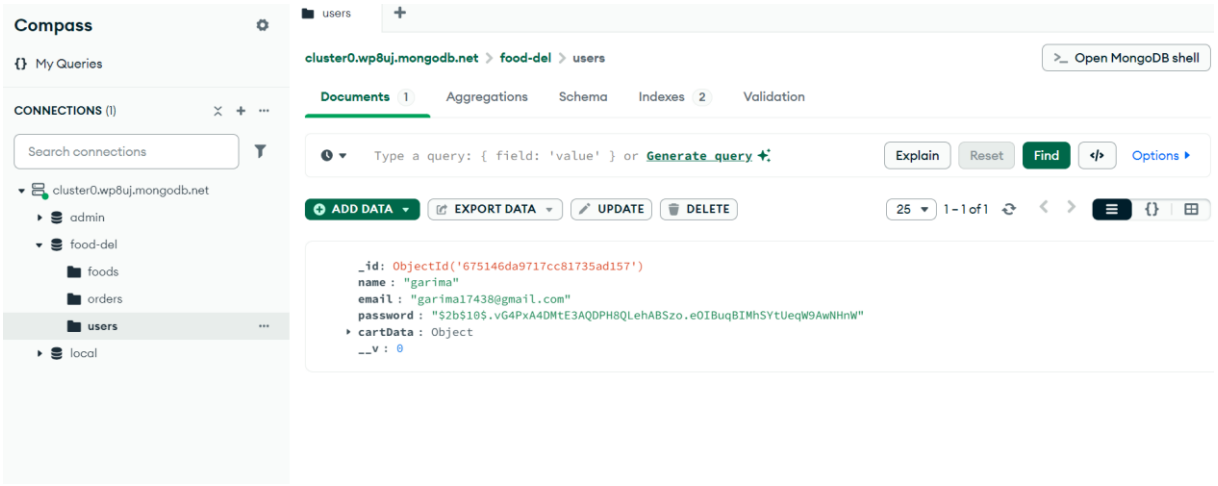
Our connection :



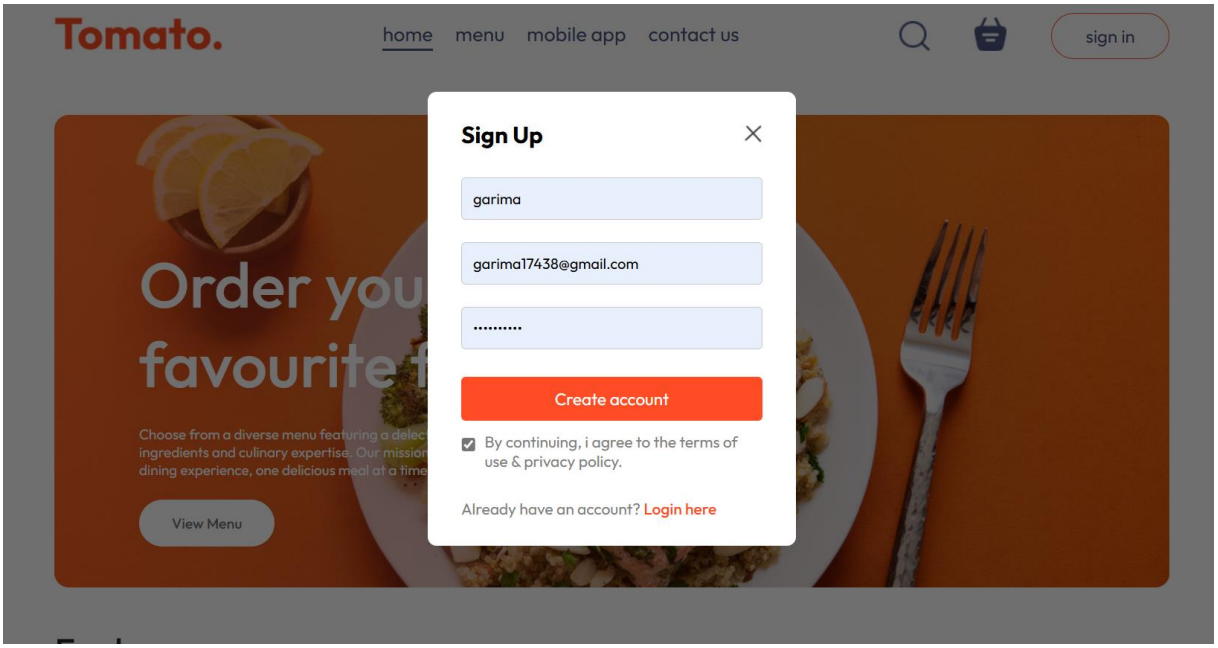

Compass
My Queries
CONNECTIONS (1)
Search connections
cluster0.wp8uj.mongodb.net
admin
food-del
foods
orders
users
local
foods
cluster0.wp8uj.mongodb.net > food-del > foods
Open MongoDB shell
Documents 27
Aggregations
Schema
Indexes 1
Validation
Type a query: { field: 'value' } or Generate query
Explain
Reset
Find
Options
ADD DATA
EXPORT DATA
UPDATE
DELETE
25
1 - 25 of 27
_id: ObjectId('675131388b6c6ecd8839e86d')
name : "Green Salad"
description : "This rich, healthy Indian salad is rich in antioxidants and vitamins..."
price : 149
image : "1733374264246food_1.png"
category : "Salad"
__v : 0
_id: ObjectId('675131ce8b6c6ecd8839e86f')
name : "Lettuce Salad"
description : "This salad is healthy salad or a side salad snack"
price : 199
image : "1733374414533food_2.png"
category : "Salad"
__v : 0
_id: ObjectId('6751324a8b6c6ecd8839e871')


Compass
My Queries
CONNECTIONS (1)
Search connections
cluster0.wp8uj.mongodb.net
admin
food-del
foods
orders
users
local
foods
cluster0.wp8uj.mongodb.net > food-del > foods
Open MongoDB sh
Documents 27
Aggregations
Schema
Indexes 1
Validation
Type a query: { field: 'value' } or Generate query
Explain
Reset
Find
Options
ADD DATA
EXPORT DATA
UPDATE
DELETE
25
1 - 25 of 27
_id: ObjectId('675134178b6c6ecd8839e876')
name : "Lasagna Rolls"
description : "This is snack that you will love to have, any time...."
price : 99
image : "1733374999607food_5.png"
category : "Rolls"
__v : 0
_id: ObjectId('675135838b6c6ecd8839e879')
name : "Mexican Rolls"
description : "best roll ever ...this is soo yummy!!"
price : 159
image : "1733375363238food_6.png"
category : "Rolls"
__v : 0

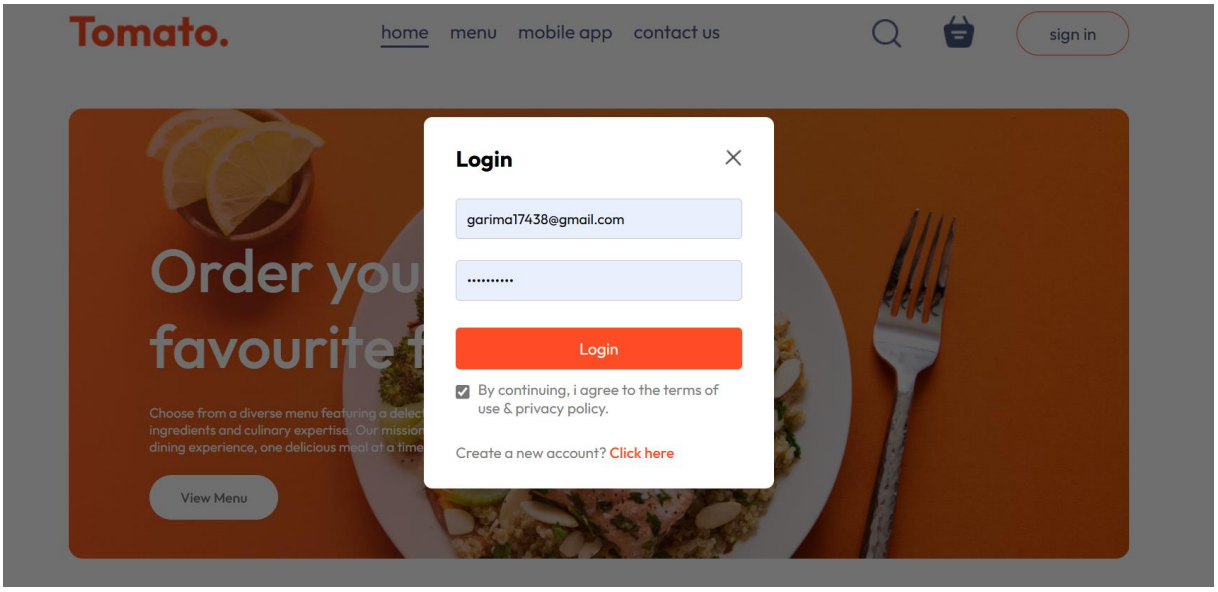

Compass
My Queries
CONNECTIONS (1)
Search connections
cluster0.wp8uj.mongodb.net
admin
food-del
foods
orders
users
local
orders
cluster0.wp8uj.mongodb.net > food-del > orders
Open MongoDB shell
Documents 2
Aggregations
Schema
Indexes 1
Validation
Type a query: { field: 'value' } or Generate query
Explain
Reset
Find
Options
ADD DATA
EXPORT DATA
UPDATE
DELETE
25
1 - 2 of 2
_id: ObjectId('675147369717cc81735ad15c')
userId : "675146da9717cc81735ad157"
items : Array (1)
amount : 249
address : Object
status : "Food Processing"
date : 2024-12-05T06:20:20.361+00:00
payment : false
__v : 0
_id: ObjectId('675147389717cc81735ad15f')
userId : "675146da9717cc81735ad157"
items : Array (1)
amount : 249
address : Object
status : "Out for delivery"
date : 2024-12-05T06:20:20.361+00:00
payment : false
__v : 0


Compass
My Queries
CONNECTIONS (1)
Search connections
cluster0.wp8uj.mongodb.net
admin
food-del
foods
orders
users
local
users
cluster0.wp8uj.mongodb.net > food-del > users
Open MongoDB shell
Documents 1
Aggregations
Schema
Indexes 2
Validation
Type a query: { field: 'value' } or Generate query
Explain
Reset
Find
Options
ADD DATA
EXPORT DATA
UPDATE
DELETE
25
1 - 1 of 1
_id: ObjectId('675146da9717cc81735ad157')
name : "garima"
email : "garima17438@gmail.com"
password : "$2b$10$.vG4PxA4DMtE3AQDPH8QLehABSzo.eOIBuqBIMhSYtUeqW9AwNHnW"
cartData : Object
__v : 0

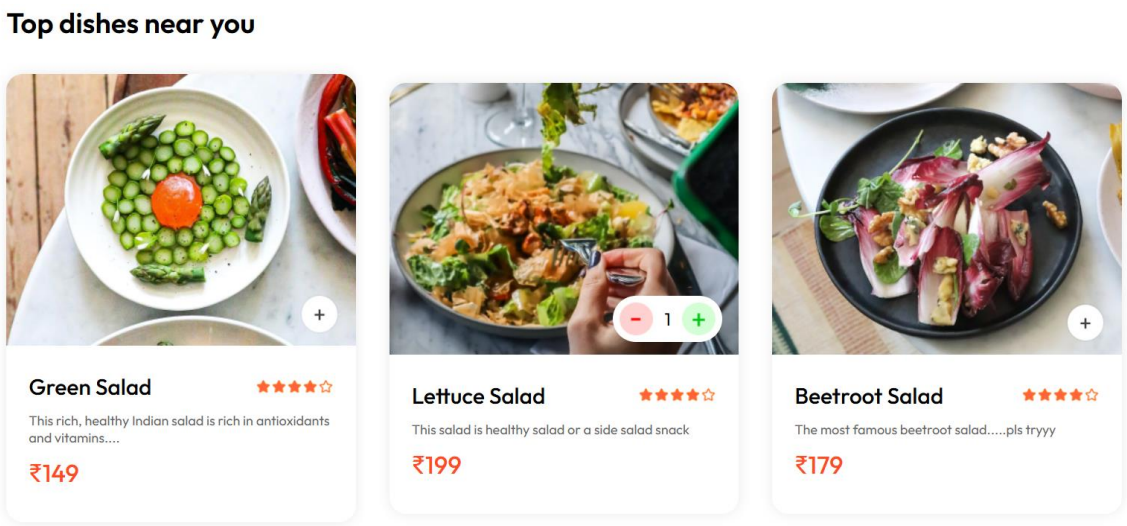
Sign in:

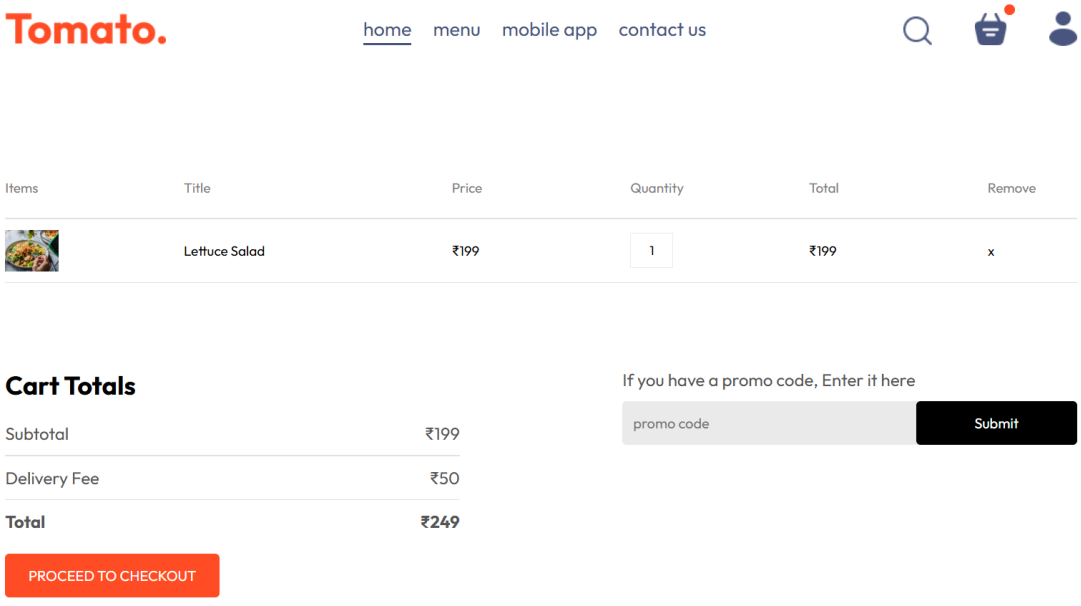

Tomato.
home
menu
mobile app
contact us
sign in
Sign Up
garima
garima17438@gmail.com
Create account
By continuing, i agree to the terms of use & privacy policy.
Already have an account? Login here
Order you
favourite
Choose from a diverse menu featuring a delec
ingredients and culinary expertise. Our mission
dining experience, one delicious meal at a time
View Menu

Login:

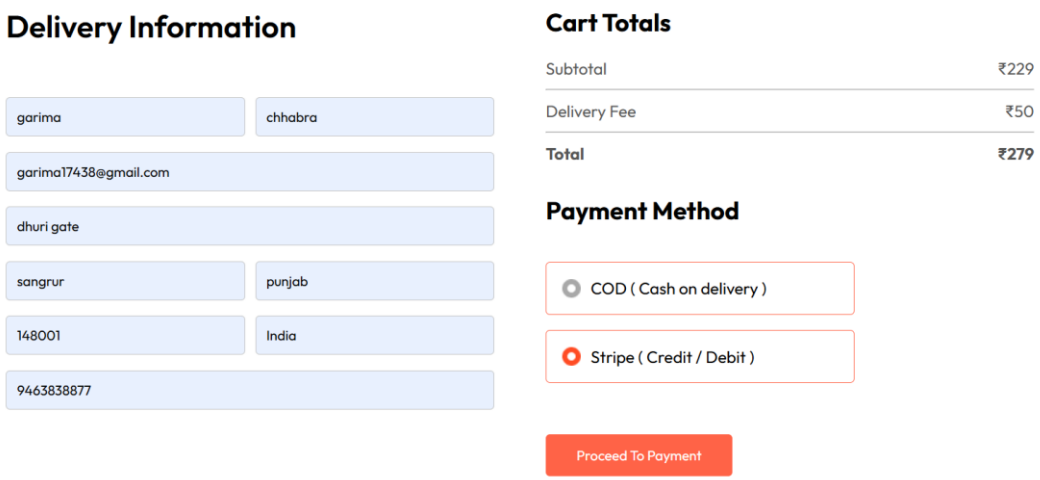

Tomato.
home
menu
mobile app
contact us
sign in
Login
garima17438@gmail.com
Login
By continuing, i agree to the terms of use & privacy policy.
Create a new account? Click here
Order you
favourite
Choose from a diverse menu featuring a delec
ingredients and culinary expertise. Our mission
dining experience, one delicious meal at a time
View Menu

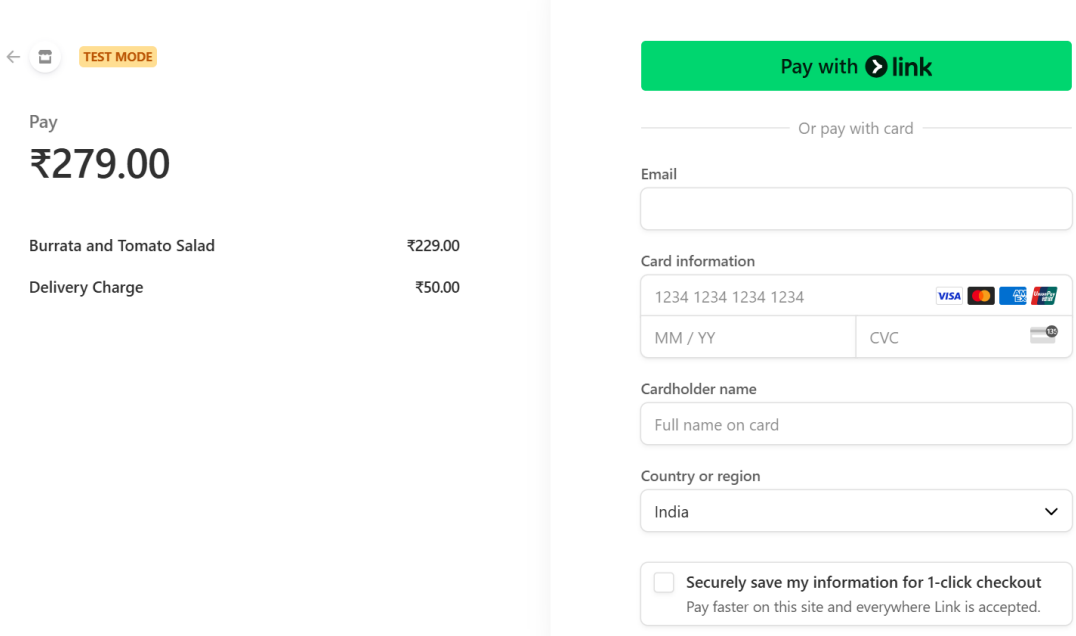
Snapshots while add to cart:

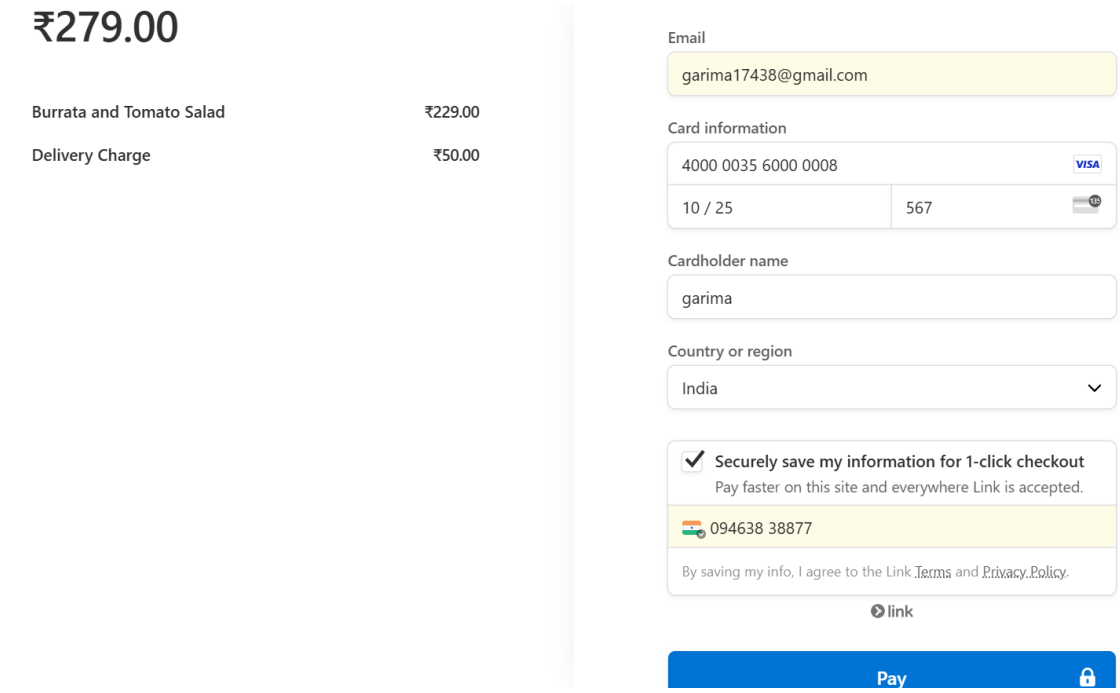




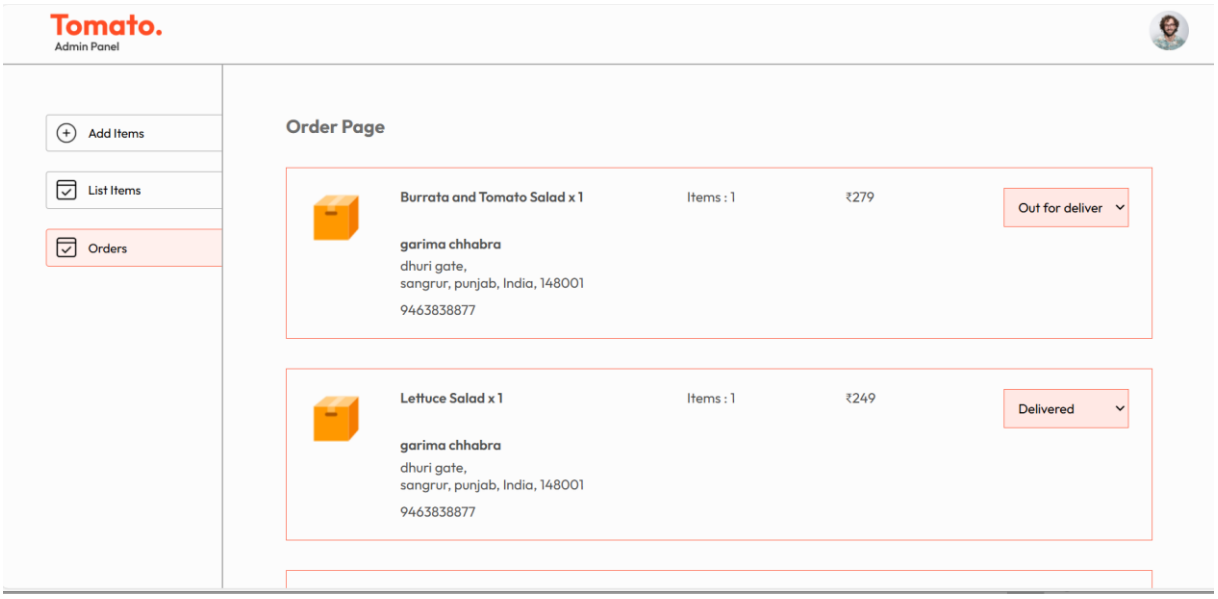
Food item delivery and payment:

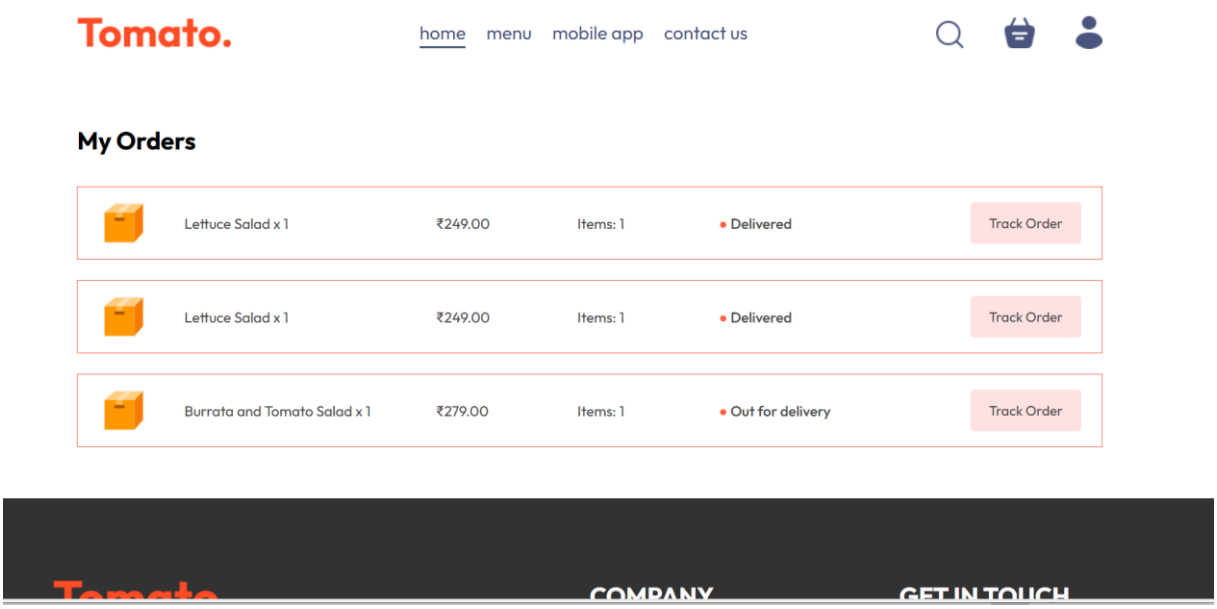






Delivery Status of the food we ordered:





5.2 Testing

Software testing is the process of evaluation a software item to detect differences between given input and expected output. Also to assess the feature of a software item. Testing assesses the quality of the product. Software testing is a process that should be done during the development process. In other words software testing is a verification and validation process.

Verification

Verification is the process to make sure the product satisfies the conditions imposed at the start of the development phase. In other words, to make sure the product behaves the way we want it to.

Validation

Validation is the process to make sure the product satisfies the specified requirements at the end of the development phase. In other words, to make sure the product is built as per customer requirements.

Basics of software testing

There are two basics of software testing: black box testing and white box testing.

Black box Testing

Black box testing is a testing technique that ignores the internal mechanism of the system and focuses on the output generated against any input and execution of the system. It is also called functional testing.

White box Testing

White box testing is a testing technique that takes into account the internal mechanism of a system. It is also called structural testing and glass box testing.

Black box testing is often used for validation and white box testing is often used for verification.

Types of testing

There are many types of testing like

- Unit Testing
- Integration Testing
- Functional Testing
- System Testing
- Stress Testing
- Performance Testing

- Usability Testing
- Acceptance Testing
- Regression Testing
- Beta Testing

❖ Unit Testing

Unit testing is the testing of an individual unit or group of related units. It falls under the class of white box testing. It is often done by the programmer to test that the unit he/she has implemented is producing expected output against given input.

❖ Integration Testing

Integration testing is testing in which a group of components are combined to produce output. Also, the interaction between software and hardware is tested in integration testing if software and hardware components have any relation. It may fall under both white box testing and black box testing.

❖ Functional Testing

Functional testing is the testing to ensure that the specified functionality required in the system requirements works. It falls under the class of black box testing.

❖ System Testing

System testing is the testing to ensure that by putting the software in different environments (e.g., Operating Systems) it still works. System testing is done with full system implementation and environment. It falls under the class of black box testing.

❖ **Stress Testing**

Stress testing is the testing to evaluate how system behaves under unfavorable conditions. Testing is conducted at beyond limits of the specifications. It falls under the class of black box testing.

❖ **Performance Testing**

Performance testing is the testing to assess the speed and effectiveness of the system and to make sure it is generating results within a specified time as in performance requirements. It falls under the class of black box testing.

❖ **Usability Testing**

Usability testing is performed to the perspective of the client, to evaluate how the GUI is user friendly? How easily can the client learn? After learning how to use, how proficiently can the client perform? How pleasing is it to use its design? This falls under the class of black box testing.

❖ **Acceptance Testing**

Acceptance testing is often done by the customer to ensure that the delivered product meets the requirements and works as the customer expected. It falls under the class of black box testing.

❖ **Regression Testing**

Regression testing is the testing after modification of a system, component, or a group of related units to ensure that the modification is working correctly and is not damaging or imposing other modules to produce unexpected results. It falls under the class of black box testing.

❖ Beta Testing

Beta testing is the testing which is done by end users, a team outside development, or publicly releasing full pre-version of the product which is known as beta version. The aim of beta testing is to cover unexpected errors. It falls under the class of black box testing.

❖ Extent of Software Testing

The basic job of software testing is to identify errors in order to reveal and spot it. The extent of software testing consists of implementation of that code in different domain and also to look at the features of the code does the software do what it is should be done and methods respect to the condition. It is proposed to begin testing from the first phase of the software development. This is not only aids in correcting the faults earlier to the last step, but also decrease the reworking of getting errors in the first step every time. It saves time as well as cost. It is a continuous method, which is probably nonstop but has to be stopped anywhere, for the need of time and resources. The basic need of the testing is to provide best quality product without taking so much time and money.

CHAPTER-6

CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

In conclusion, the food app built with the MERN stack provides an easy and reliable way for users to discover and order food. Using MongoDB, Express, React, and Node.js, the app ensures a smooth experience from browsing menus to placing orders. The React front-end makes it simple for users to navigate, while the backend ensures fast and secure order processing. The app also includes real-time features and secure payment options, making it both convenient and safe to use. Overall, this food app is designed to meet the needs of customers and restaurant owners, offering a great solution in the busy world of food delivery.

- There can be types of payment method like if any user wants to pay online then he can pay by using his card details or by e-banking.
- We will show the users about all the status of his order when it is packed, and at what time and date it is delivered and on what date it will be delivered to the users.

6.2 Future Scope

In the future we can make changes in our website. The changes can be:

We have to work on a review option in our project and an order tracking module because now the delivery of food orders is not tracked. Going forward, the delivery of our food orders will also be tracked in the future. This is our future scope regarding the project.

Several areas for future work might be highlighted for improvement. The app, for example, may be upgraded to incorporate other features such as a rating system for restaurants and drivers, or the ability to follow the delivery truck in real-time and track delivery status.

Furthermore, the app's speed and scalability might be improved by incorporating caching systems or load-balancing approaches. Finally, the creation of a food delivery application utilising the MERN stack resulted in a functioning and efficient system that fits the demands of both users and restaurants. With a user-friendly design and real-time delivery progress information, the programme provides a quick and simplified method to order and transport meals. Modern technologies like MongoDB, React JS, and Node JS were used to provide a scalable and adaptable solution that can be easily customised and expanded in the future.

References

https://www.google.com/

https://www.pexels.com/

https://unsplash.com/

https://www.mongodb.com/products/platform/atlas-database

https://stripe.com/in

https://www.youtube.com/

https://www.zomato.com/